

Python Practice Workbook

Module 4: Multivariate & Volatility

VAR, ARCH, GARCH, Conditional Heteroscedasticity

Time Series Analytics T6725 — M.Sc. Data Science Sem IV

Symbiosis International (Deemed) University

Primary Dataset: BTC-USD — Supporting: MRF.NS, GC=F

Standard Setup Block — Run this first in every Colab session

```
!pip install yfinance statsmodels arch -q
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Problem Distribution

Difficulty	Count	Types Used
EASY	10	FIB, FTE, MCQ, PTO, SA
MEDIUM	12	FIB, FTE, GOR, MTF, MCQ, WFS
HARD	12	FIB, FTE, MCQ, WFS, PTO, SA
EXAM	6	Multi-part
TOTAL	40	All 8 types

Section A

Easy Problems — M4.001 to M4.010

M4.001 Fill in the Blank **EASY**

Topic: ARCH effect test — Engle's LM test

Fill in the **four blanks** to run Engle's ARCH LM test on BTC log returns and detect volatility clustering.

```

1 from arch.univariate import _____ # ARCH test function
   lives here
2
3 btc      = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
4 returns = np.log(btc["Close"]).diff().dropna() * 100 #
   scale to percentage
5
6 # Engle's LM test: H0 = No ARCH effects
7 result  = arch_lm_test(returns, nlags=_____)
8
9 print(f"LM Statistic : {result.stat:.4f}")
10 print(f"p-value      : {result._____.6f}")
11 print(f"Lags tested  : {result.lags}")
12
13 if result.pvalue < _____:
14     print("ARCH effects DETECTED -- GARCH model is
   appropriate")
15 else:
16     print("No significant ARCH effects")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from arch.univariate import arch_lm_test # Engle's LM test
7
8 btc      = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
9 returns = np.log(btc["Close"]).diff().dropna() * 100 #
   x100 -> percentage
10
11 # H0: No ARCH effects (constant conditional variance)
12 result  = arch_lm_test(returns, nlags=10) # test up to
   10 lags
13

```

```

14 print(f"LM Statistic : {result.stat:.4f}")
15 print(f"p-value      : {result.pvalue:.6f}")    # attribute
    is .pvalue
16 print(f"Lags tested  : {result.lags}")
17
18 if result.pvalue < 0.05:    # standard 5% significance level
19     print("ARCH effects DETECTED -- GARCH model is
        appropriate")
20 else:
21     print("No significant ARCH effects")

```

② Plain English

Engle's LM test checks whether squared returns are autocorrelated — if large squared returns tend to follow large squared returns, that's volatility clustering (ARCH effect). The test regresses squared residuals on their own lags; a significant F-statistic (small p-value) means we reject constant variance and need a GARCH-type model. For BTC, p will be essentially zero — the volatility clustering is unmistakable.

③ Common Mistake

Testing the raw returns instead of squared returns. ARCH effects are about variance dynamics, not mean dynamics — the Ljung-Box test on raw returns tests mean autocorrelation, while Engle's LM test on squared returns tests variance autocorrelation. Running Ljung-Box on raw BTC returns may not show significance, but Engle's LM on squared returns almost certainly will.

④ Real World Link

BTC's ARCH effects are among the strongest of any asset class. During calm periods, daily BTC moves might be 1–2%; during volatile regimes, the same model might see 8–12% daily moves for weeks. GARCH quantifies this switching behaviour and is the foundation of BTC options pricing.

M4.002 Fill in the Blank **EASY**

Topic: Fitting a basic GARCH(1,1) model

Fill in the blanks to fit the most common GARCH specification to MRF returns.

```

1 from arch import arch_model
2
3 mrf      = yf.download("MRF.NS", start="2019-01-01",
    end="2024-12-31")
4 returns = np.log(mrf["Close"]).diff().dropna() * 100
5
6 model = arch_model(
7     returns,
8     mean = "_____",    # constant mean
    specification
9     vol  = "_____",    # GARCH volatility model
10    p    = _____,  # ARCH lag order

```

```

11         q      = -----      # GARCH lag order
12     )
13
14     fit = model.fit(dis="off")
15     print(fit.summary())
16
17     cond_vol = fit.conditional_volatility
18     print(f"\nMean conditional vol : {cond_vol.mean():.4f}%
19           per day")
20     print(f"Max conditional vol : {cond_vol.max():.4f}% per
21           day")

```

Solution

① Correct Code

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import numpy as np
4  import warnings
5  warnings.filterwarnings('ignore')
6  from arch import arch_model
7
8  mrf      = yf.download("MRF.NS", start="2019-01-01",
9                        end="2024-12-31")
10
11  returns = np.log(mrf["Close"]).diff().dropna() * 100
12
13  model = arch_model(
14      returns,
15      mean = "Constant", # mean = omega (intercept
16                        # only)
17      vol  = "GARCH",    # GARCH variance equation
18      p    = 1,          # ARCH term: lag-1 squared
19                        # shock
20      q    = 1           # GARCH term: lag-1
21                        # variance
22  )
23
24  fit = model.fit(dis="off") # dis="off" suppresses
25                        # iteration output
26  print(fit.summary())
27
28  cond_vol = fit.conditional_volatility
29  print(f"\nMean conditional vol : {cond_vol.mean():.4f}%
30        per day")
31  print(f"Max conditional vol : {cond_vol.max():.4f}% per
32        day")

```

② Plain English

GARCH(1,1) says today's variance (σ_t^2) equals a long-run baseline (ω) plus α_1 times yesterday's squared shock (how big was yesterday's move?) plus β_1 times yesterday's variance (how volatile has it been recently?). The conditional volatility series extracted by `fit.conditional_volatility` tells you the model's day-by-day risk estimate.

③ Common Mistake

Forgetting to multiply returns by 100 before fitting. The `arch` library works best with percentage returns (order of magnitude ~ 1) rather than decimal returns (order of magnitude ~ 0.01). With decimal returns, the estimated ω becomes extremely small and numerical precision issues arise in the optimisation.

④ Real World Link

GARCH(1,1) was introduced by Tim Bollerslev in 1986 and remains the workhorse of volatility modelling four decades later. Every bank's Value-at-Risk calculation, every option pricing desk, and every risk management system uses some variant of GARCH. For MRF, the conditional volatility spike in March 2020 directly translates to wider bid-ask spreads and higher option premiums.

M4.003 Multiple Choice **EASY**

Topic: GARCH(1,1) — variance equation interpretation

The GARCH(1,1) variance equation is $\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$. What does a large value of β (close to 1) imply about volatility?

- (A) Volatility is highly responsive to new shocks — every large return immediately and permanently raises variance
- (B) Volatility is highly persistent — once elevated, it decays slowly back to the long-run level
- (C) The long-run variance is large — the unconditional variance is approximately $\omega/(1 - \beta)$
- (D) The model is non-stationary — GARCH requires $\beta < 1$ for stability

Solution

① Correct Answer: (B)

② Plain English

β is the GARCH persistence parameter. When $\beta \approx 0.9$ (typical for financial assets), today's variance is 90% determined by yesterday's variance — the process has long memory and volatility decays slowly. A shock to volatility takes many periods to dissipate. This is why we observe “volatility regimes” — after a crash like COVID, elevated BTC volatility persists for weeks or months, not just days. α (not β) controls how much new shocks impact variance immediately.

③ Common Mistake

Choosing (C) — the long-run variance formula is $\omega/(1-\alpha-\beta)$, not $\omega/(1-\beta)$. Both α and β together determine persistence. Also choosing (D) — GARCH requires $\alpha + \beta < 1$ for covariance stationarity, and β can be close to 1 as long as $\alpha + \beta < 1$.

④ Real World Link

BTC typically shows $\beta \approx 0.85\text{--}0.92$ and $\alpha \approx 0.08\text{--}0.12$, giving $\alpha + \beta \approx 0.95\text{--}0.97$. This near-unit-root persistence means BTC volatility shocks from a major event can persist for 3–6 months. This has direct implications for crypto derivatives: long-dated options remain expensive long after the triggering event.

M4.004 Multiple Choice **EASY**

Topic: VAR model — what it models

A VAR(1) model for two variables (BTC returns and Gold returns) is:

$$\text{BTC}_t = c_1 + a_{11} \cdot \text{BTC}_{t-1} + a_{12} \cdot \text{Gold}_{t-1} + \varepsilon_{1,t}$$

$$\text{Gold}_t = c_2 + a_{21} \cdot \text{BTC}_{t-1} + a_{22} \cdot \text{Gold}_{t-1} + \varepsilon_{2,t}$$

What does a significant coefficient a_{12} indicate?

- (A) Gold returns at time t predict BTC returns at time t (contemporaneous correlation)
- (B) Gold returns at time $t - 1$ have predictive power for BTC returns at time t (lagged cross-prediction)
- (C) BTC and Gold have the same unconditional variance
- (D) The VAR model is misspecified and should be replaced with separate ARIMA models

Solution

① Correct Answer: (B)

② Plain English

VAR models capture lead-lag relationships — each variable is regressed on its own lags AND the lags of all other variables. A significant a_{12} means yesterday's Gold return has predictive power for today's BTC return — Gold “leads” BTC. Note: VAR uses only lagged values, so there's no contemporaneous regression (which would cause endogeneity).

③ Common Mistake

Confusing a_{12} (Gold→BTC) with a_{21} (BTC→Gold). In a 2×2 VAR matrix, the off-diagonal elements capture cross-variable prediction. If both a_{12} and a_{21} are significant, the relationship is bidirectional — BTC and Gold mutually Granger-cause each other.

④ Real World Link

In practice, the BTC-Gold cross-prediction is weak at the daily frequency — both markets are nearly efficient. The VAR is more useful at monthly or weekly frequency, where macro factors (inflation expectations, dollar weakness) create genuine lead-lag relationships through the investor decision channel.

M4.005 Predict the Output **EASY**

Topic: GARCH parameter constraints

Without running this code, predict what will happen and why.

```

1 from arch import arch_model
2 import numpy as np
3
4 np.random.seed(42)
5 returns = np.random.randn(500) * 2 # iid data -- no
   genuine ARCH effects
6
7 model = arch_model(returns, mean="Constant", vol="GARCH",
   p=1, q=1)
8 fit = model.fit(dis="off")
9
10 omega = fit.params["omega"]
11 alpha = fit.params["alpha[1]"]
12 beta = fit.params["beta[1]"]
13
14 print(f"omega : {omega:.6f}")
15 print(f"alpha : {alpha:.6f}")
16 print(f"beta : {beta:.6f}")
17 print(f"alpha + beta : {alpha + beta:.6f}")
18 print(f"Stationary? : {alpha + beta < 1}")

```

Predict: (i) will all three parameters be positive? (ii) will $\alpha + \beta$ be less than 1? (iii) approximately what will ω be?

Solution

① Predicted Output

```

omega : [small positive, approximately 0.1--2.0]
alpha : [small positive, approximately 0.05--0.15]
beta : [moderate positive, approximately 0.7--0.9]
alpha + beta : [value < 1, approximately 0.75--0.95]
Stationary? : True

```

```

1 # Reasoning:
2 # Data is iid  $N(0,4)$  -- pure white noise with CONSTANT

```

```

    variance
3  # There are NO genuine ARCH effects -- true model:
    alpha=0, beta=0
4  #
5  # However, arch library constrains:
6  #   omega > 0, alpha >= 0, beta >= 0 (non-negativity)
7  #   alpha + beta < 1 (stationarity -- enforced by
    optimiser)
8  #
9  # For iid data, MLE tries to fit alpha~0, beta~0 but
    numerical
10 # optimisation pushes estimates to small but positive
    values.
11 #
12 # omega ~ variance x (1 - alpha - beta)
13 # True variance = 4, so if alpha+beta~0.9: omega ~ 4 x 0.1
    = 0.4
14 #
15 # Stationary = True because arch enforces alpha+beta < 1

```

② Plain English

Even for data with no genuine ARCH effects, GARCH will fit positive parameters — the optimiser just doesn't know the true model is iid. The non-negativity and stationarity constraints ($\alpha + \beta < 1$) are enforced computationally, ensuring the fitted model is always a valid GARCH process. The key check is whether the ARCH LM test on residuals still shows ARCH effects after fitting — for iid data, it won't.

③ Common Mistake

Expecting $\alpha = 0$ and $\beta = 0$ exactly for iid data. Maximum likelihood estimators are random variables — they won't land exactly at the true values in any finite sample. The estimates will be small but non-zero, and that's statistically normal.

④ Real World Link

This experiment is useful for calibration: if you fit GARCH to a series and get $\alpha + \beta > 0.99$ or β close to 1, that means strong volatility persistence. For BTC, expect $\alpha + \beta \approx 0.96$ — near-integrated volatility process.

M4.006 Short Answer **EASY**

Topic: ARCH vs GARCH — difference

In 2 sentences, explain the key difference between ARCH(q) and GARCH(p,q) models, and state why GARCH(1,1) is almost always preferred over ARCH(q) for financial returns.

Solution**① Model Answer**

ARCH(q) models today's variance as a function of only the past q squared returns (shocks), while GARCH(p, q) adds p lagged conditional variances as additional predictors — essentially making GARCH an ARMA model for variance rather than just an AR model. GARCH(1,1) is preferred because it achieves the parsimony of just 3 variance parameters (ω, α, β) while capturing the same long-memory persistence that would require a high-order ARCH(q) with many parameters, making it both more efficient to estimate and less prone to overfitting.

② Common Mistake

Saying GARCH is “more complex” than ARCH. In terms of parameter count, GARCH(1,1) has fewer parameters than ARCH(10), which is what you'd need to match its persistence. The relationship is analogous to AR(p) vs ARMA(p, q) for mean models — adding the MA component (lagged variances in GARCH) achieves parsimony by capturing long-run effects with fewer parameters.

③ Real World Link

The dominance of GARCH(1,1) in practice is remarkable — a 1986 model with 3 variance parameters still outperforms or matches most alternatives on real financial data. JPMorgan's RiskMetrics system (which powered global bank VaR calculations for decades) is essentially an integrated GARCH(1,1) with $\alpha = 0.06$, $\beta = 0.94$, $\omega = 0$. Understanding GARCH(1,1) deeply is understanding the foundation of institutional risk management.

M4.007 Find the Error EASY

Topic: VAR model — wrong variable format

This code crashes immediately. Find and fix the error.

```

1 from statsmodels.tsa.vector_ar.var_model import VAR
2
3 btc = yf.download("BTC-USD", start="2020-01-01",
4   end="2024-12-31")
5
6 gold = yf.download("GC=F", start="2020-01-01",
7   end="2024-12-31")
8
9 btc_ret = np.log(btc["Close"]).diff().dropna()
10 gold_ret = np.log(gold["Close"]).diff().dropna()
11
12 # ERROR: passing two separate Series instead of one
13   DataFrame
14
15 model = VAR([btc_ret, gold_ret]) # ERROR
16 fit = model.fit(maxlags=5, ic="aic")
17 print(fit.summary())

```

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import warnings; warnings.filterwarnings('ignore')
4 from statsmodels.tsa.vector_ar.var_model import VAR
5
6 btc = yf.download("BTC-USD", start="2020-01-01",
7                 end="2024-12-31")
8
9 gold = yf.download("GC=F", start="2020-01-01",
10                 end="2024-12-31")
11
12 btc_ret = np.log(btc["Close"]).diff().dropna()
13 gold_ret = np.log(gold["Close"]).diff().dropna()
14
15 # FIX: VAR requires a single DataFrame with all variables
16 #      as columns
17 # Align on common trading dates first (inner join -- BTC
18 # trades weekends!)
19 combined = btc_ret.to_frame("BTC").join(
20             gold_ret.to_frame("Gold"), how="inner"
21         ).dropna()
22
23 print(f"Combined shape: {combined.shape}") # should be
24                                             (n, 2)
25
26 # Now pass the DataFrame -- not a list of Series
27 model = VAR(combined)
28 fit = model.fit(maxlags=5, ic="aic")
29 print(fit.summary())

```

② Plain English

The VAR model requires all variables to be columns in a single aligned DataFrame — it can't accept a Python list of separate Series because it needs to know the dates align and handle any missing values consistently. The inner join ensures BTC and Gold observations are paired on the exact same trading dates — critical because BTC trades weekends while Gold futures don't.

③ Common Mistake

Forgetting the inner join before concatenating. BTC has ~250 extra weekend observations per year compared to Gold. If you just `pd.concat([btc_ret, gold_ret], axis=1)`, the Gold column has NaN on weekends — and VAR will either crash or produce biased estimates. The inner join keeps only dates where both have observations.

④ Real World Link

In a real multi-asset VAR for a portfolio containing BTC, MRF, and Gold, the date alignment step is crucial. Institutional systems often resample everything to

weekly frequency (Friday closing prices) to avoid the weekend-trading asymmetry. The choice of frequency affects which lead-lag relationships you detect.

M4.008 Fill in the Blank **EASY**

Topic: GARCH volatility forecasting

Fill in the blanks to generate a 10-day volatility forecast from a fitted GARCH(1,1) model.

```

1 from arch import arch_model
2
3 btc      = yf.download("BTC-USD", start="2019-01-01",
4                       end="2024-12-31")
5 returns = np.log(btc["Close"]).diff().dropna() * 100
6
7 model = arch_model(returns, mean="Constant", vol="GARCH",
8                   p=1, q=1)
9 fit    = model.fit(dispatch="off")
10
11 # Generate 10-step volatility forecast
12 forecast = fit._____(horizon=_____, reindex=False)
13
14 # Extract variance forecasts
15 var_forecast = forecast.variance._____ [0] # last row
16 vol_forecast = np._____(var_forecast)        # convert
17           variance to vol
18
19 print("10-Day Volatility Forecast (% per day):")
20 for h, vol in enumerate(vol_forecast, 1):
21     print(f"  Day {h:2d}: {vol:.4f}%")
22
23 # Annualise: multiply daily vol by sqrt(252)
24 annual_vol = vol_forecast[-1] * np.sqrt(_____)
25 print(f"\nAnnualised vol at day 10: {annual_vol:.2f}%")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import warnings; warnings.filterwarnings('ignore')
4 from arch import arch_model
5
6 btc      = yf.download("BTC-USD", start="2019-01-01",
7                       end="2024-12-31")
8 returns = np.log(btc["Close"]).diff().dropna() * 100
9
10 model = arch_model(returns, mean="Constant", vol="GARCH",
11                   p=1, q=1)
12 fit    = model.fit(dispatch="off")
13
14 # Generate 10-step volatility forecast
15 forecast = fit.forecast(horizon=10, reindex=False)
16
17 # Extract variance forecasts
18 var_forecast = forecast.variance.iloc[-1] # last row
19 vol_forecast = np.sqrt(var_forecast)      # convert
20           variance to vol
21
22 print("10-Day Volatility Forecast (% per day):")
23 for h, vol in enumerate(vol_forecast, 1):
24     print(f"  Day {h:2d}: {vol:.4f}%")
25
26 # Annualise: multiply daily vol by sqrt(252)
27 annual_vol = vol_forecast[-1] * np.sqrt(252)
28 print(f"\nAnnualised vol at day 10: {annual_vol:.2f}%")

```

```

9  model = arch_model(returns, mean="Constant", vol="GARCH",
    p=1, q=1)
10 fit    = model.fit(dispatch="off")
11
12 # Generate 10-step volatility forecast
13 forecast = fit.forecast(horizon=10, reindex=False) #
    .forecast() method
14
15 # Extract variance forecasts -- row 0 (only row when
    reindex=False)
16 var_forecast = forecast.variance.iloc[0]
17 vol_forecast = np.sqrt(var_forecast) # std dev =
    sqrt(variance)
18
19 print("10-Day Volatility Forecast (% per day):")
20 for h, vol in enumerate(vol_forecast, 1):
21     print(f"  Day {h:2d}: {vol:.4f}%")
22
23 # Annualise: sqrt of time rule, 252 trading days per year
24 annual_vol = vol_forecast.iloc[-1] * np.sqrt(252)
25 print(f"\nAnnualised vol at day 10: {annual_vol:.2f}%")

```

② Plain English

GARCH forecasts variance, not volatility — take the square root to get the more interpretable standard deviation. The `forecast.variance` object contains a matrix of forecasted variances at each horizon. With `reindex=False`, the output has one row and `horizon` columns (days 1 through 10). The 10-day forecast will converge toward the unconditional variance as the horizon extends.

③ Common Mistake

Converting to annual volatility by multiplying by 252 instead of $\sqrt{252}$ — many students square the daily vol before annualising, which gives variance, not volatility. Always annualise vol as `daily_std × sqrt(252)`.

④ Real World Link

This 10-day volatility forecast is the input to BTC options pricing. If GARCH forecasts 4.2% daily vol over the next 10 days, the 10-day standard deviation is $4.2\% \times \sqrt{10} \approx 13.3\%$ — meaning the market expects BTC to be within $\pm 13.3\%$ of current price with $\sim 68\%$ probability over the next 10 days.

M4.009 Multiple Choice EASY

Topic: Granger causality — null hypothesis

You run a Granger causality test from BTC returns to Gold returns (testing whether BTC “Granger-causes” Gold). The p-value is 0.32. What is the correct conclusion?

- (A) BTC Granger-causes Gold — reject H_0 because $p = 0.32$ is relatively small
- (B) Fail to reject H_0 — BTC’s past returns do not significantly improve predictions of Gold’s returns beyond Gold’s own history

- (C) Gold Granger-causes BTC — the test result applies in both directions simultaneously
- (D) BTC and Gold are cointegrated — the p-value indicates a long-run relationship

Solution

① Correct Answer: (B)

② Plain English

Granger causality H_0 says “X does NOT Granger-cause Y” — meaning X’s past values add no predictive information for Y beyond Y’s own past. A p-value of 0.32 is well above 0.05, so we fail to reject H_0 — BTC’s past returns do not significantly improve Gold price predictions. This means in a VAR for Gold, BTC’s lagged terms are statistically redundant.

③ Common Mistake

Confusing Granger causality (predictive) with true causality (structural). “BTC does NOT Granger-cause Gold” doesn’t mean BTC has no economic relationship with Gold — it means BTC’s past provides no extra predictive signal for Gold returns at this frequency and lag length.

④ Real World Link

In practice, the BTC-Gold Granger causality result is mixed and time-varying. During crypto-correlated risk-off episodes, they might show mutual Granger causality. During normal periods, neither significantly leads the other at daily frequency. This is exactly why VAR order selection matters.

M4.010 Short Answer EASY

Topic: Volatility clustering — definition and evidence

In 2 sentences, define volatility clustering and describe one visual test you can perform in Python to detect it in BTC daily returns.

Solution

① Model Answer

Volatility clustering is the empirical observation that large price changes (of either sign) tend to be followed by large price changes, and small changes tend to be followed by small changes — meaning the conditional variance of returns is autocorrelated and predictable. To detect it visually, plot the squared log returns of BTC over time (`(returns**2).plot()`) and look for distinct “clusters” of elevated values alternating with calm periods, or equivalently plot the ACF of squared returns (`plot_acf(returns**2, lags=40)`) and check for significant spikes indicating that today’s variance predicts tomorrow’s.

② Common Mistake

Saying “volatility clustering means returns are autocorrelated.” It doesn’t — returns themselves may be close to white noise (no mean autocorrelation), but their squares or absolute values are highly autocorrelated. Volatility clustering is a variance phenomenon, not a mean phenomenon.

③ Real World Link

BTC’s volatility clustering is visually obvious in any multi-year chart — the 2021 bull run shows daily moves of 5–15%, the 2023 consolidation shows 1–3% daily moves, and the 2024 halving period shows 3–7% moves. This alternation of “calm” and “storm” regimes is exactly what GARCH models capture quantitatively.

Section B

Medium Problems — M4.011 to M4.022

M4.011 *Fill in the Blank* MEDIUM

Topic: Fitting VAR and selecting lag order

Fill in the blanks to build a VAR model for BTC and MRF returns with automatic lag selection.

```

1 from statsmodels.tsa.vector_ar.var_model import VAR
2
3 btc = yf.download("BTC-USD", start="2020-01-01",
4   end="2024-12-31")
5 mrf = yf.download("MRF.NS", start="2020-01-01",
6   end="2024-12-31")
7
8 btc_ret = np.log(btc["Close"]).diff()
9 mrf_ret = np.log(mrf["Close"]).diff()
10
11 data = btc_ret.to_frame("BTC").join(
12     mrf_ret.to_frame("MRF"), how="_____"
13 )
14
15 print(f"Dataset shape: {data.shape}")
16
17 model = VAR(_____)
18 results = model.select_order(maxlags=_____)
19 print(results.summary())
20
21 optimal_lag = results._____ ["aic"]
22 fit = model.fit(_____)
23 print(f"\nVAR fitted with {fit.k_ar} lags")
24 print(f"Number of equations: {fit._____.shape[1]}")

```

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import warnings; warnings.filterwarnings('ignore')
4 from statsmodels.tsa.vector_ar.var_model import VAR
5
6 btc = yf.download("BTC-USD", start="2020-01-01",
7   end="2024-12-31")
8 mrf = yf.download("MRF.NS", start="2020-01-01",
9   end="2024-12-31")
10
11 btc_ret = np.log(btc["Close"]).diff()
12 mrf_ret = np.log(mrf["Close"]).diff()
13
14 data = btc_ret.to_frame("BTC").join(
15   mrf_ret.to_frame("MRF"), how="inner"    # keep
16   only shared dates
17 ).dropna()
18
19 print(f"Dataset shape: {data.shape}")
20
21 model = VAR(data)
22 results = model.select_order(maxlags=10)    # evaluate lags
23   1 through 10
24 print(results.summary())
25
26 optimal_lag = results.selected_orders["aic"]    # dict
27   keyed by criterion name
28 fit = model.fit(optimal_lag)
29 print(f"\nVAR fitted with {fit.k_ar} lags")
30 print(f"Number of equations: {fit.neqs}")    # neqs =
31   number of variables

```

② Plain English

`select_order()` fits VAR models at each lag length from 1 to `maxlags` and computes AIC, BIC, FPE, and HQIC for each — then returns the lag count minimising each criterion. For BTC and MRF daily returns, AIC typically selects a small lag (1–3) because returns are close to white noise and long lag structures overfit.

③ Common Mistake

Using `results.aic` instead of `results.selected_orders["aic"]`. The `select_order()` result object stores the optimal lag choices in a dictionary accessed by criterion name, not as direct attributes. Print `results.summary()` first to see all values.

④ Real World Link

A VAR(1) for BTC and MRF returns captures the leading question: “Does yesterday’s BTC crash predict today’s MRF decline?” For Indian equities, BTC’s influence

flows through foreign institutional investor (FII) flows into NSE-listed stocks like MRF. If the VAR detects significant cross-lagged coefficients, that's a quantifiable Granger causality channel worth investigating.

M4.012 Find the Error **MEDIUM**

Topic: GARCH stationarity violation

This code fits a GARCH model and silently produces an explosive process. Find the error and its consequence.

```

1 from arch import arch_model
2 import numpy as np
3
4 np.random.seed(1)
5 returns = np.random.randn(800) * 2
6
7 model = arch_model(returns, mean="Zero", vol="GARCH", p=1,
8                   q=1)
9 fit = model.fit(dispatch="off",
10                 options={"ftol": 1e-2,      # ERROR:
11                           loose convergence tolerance
12                           "maxiter": 5})    # ERROR: too
13                                           few iterations
14
15 alpha = fit.params["alpha[1]"]
16 beta = fit.params["beta[1]"]
17 omega = fit.params["omega"]
18
19 print(f"alpha + beta = {alpha + beta:.4f}")    # may be > 1
20 -- explosive!
21 print(f"Long-run variance = {omega / (1 - alpha -
22     beta):.4f}")    # may crash

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import numpy as np, warnings
3 warnings.filterwarnings('ignore')
4 from arch import arch_model
5
6 np.random.seed(1)
7 returns = np.random.randn(800) * 2
8
9 # ROOT CAUSE: Extremely loose convergence settings cause
10 the optimiser to stop

```



```

10 # before reaching the true maximum likelihood --
    parameters may violate constraints.
11 # alpha + beta > 1 -> explosive non-stationary GARCH
    (variance -> infinity)
12 # Long-run variance formula omega/(1-alpha-beta) has
    negative denominator -> crash
13
14 # FIX: Use defaults -- do NOT override options without a
    strong reason
15 model = arch_model(returns, mean="Zero", vol="GARCH", p=1,
    q=1)
16 fit = model.fit(dispatch="off")
17 # Default ftol ~1e-8 and maxiter ~500 -- both appropriate
18
19 omega = fit.params["omega"]
20 alpha = fit.params["alpha[1]"]
21 beta = fit.params["beta[1]"]
22
23 print(f"alpha + beta = {alpha + beta:.6f}")
24 assert alpha + beta < 1, "Non-stationary GARCH! alpha+beta
    >= 1"
25
26 lrv = omega / (1 - alpha - beta)
27 print(f"Long-run variance = {lrv:.6f} (positive = valid)")
28 print(f"Long-run std = {np.sqrt(lrv):.6f}%")
29 print(f"\nOptimisation converged: {fit.convergence_flag ==
    0}")

```

② Plain English

With `maxiter=5` and loose tolerance, the optimiser terminates after just 5 steps — far from the maximum likelihood solution. The unconstrained intermediate parameters can violate $\alpha + \beta < 1$, producing a non-stationary GARCH process where the long-run variance formula $\omega/(1 - \alpha - \beta)$ has a negative denominator. Always use default convergence settings unless you have a strong reason to override them.

③ Common Mistake

Overriding convergence settings to “speed up” model fitting without understanding the consequences. GARCH log-likelihood surfaces are not convex and can have flat regions requiring many iterations to navigate. With `maxiter=5`, you’re almost certainly not at the maximum — you’re at some arbitrary point in parameter space that may produce garbage estimates.

④ Real World Link

Non-stationary GARCH ($\alpha + \beta \geq 1$) is called IGARCH (Integrated GARCH). While some researchers argue BTC’s volatility process is near-integrated, using IGARCH requires explicit justification. Accidentally fitting IGARCH due to poor convergence settings will produce variance forecasts that grow to infinity, making them useless for risk management.

M4.013 *Get the Order Right* **MEDIUM****Topic:** GARCH model building workflow

These **8 steps** for fitting and validating a GARCH model are scrambled. Write the correct order.

- (A) Standardise residuals: $z_t = \varepsilon_t / \sigma_t$ — check for remaining ARCH effects
- (B) Download BTC data and compute log returns ($\times 100$ for percentage)
- (C) Plot squared returns and ACF of squared returns — visual ARCH check
- (D) Print model summary — inspect ω, α, β and their t-statistics
- (E) Run Engle's ARCH LM test on raw returns — confirm ARCH effects
- (F) Generate volatility forecast (e.g. 22 trading days ahead)
- (G) Specify and fit GARCH(1,1) using `arch_model()`
- (H) Verify $\alpha + \beta < 1$ — confirm GARCH stationarity condition

Solution

① **Correct Order:** B $\rightarrow$ C $\rightarrow$ E $\rightarrow$ G $\rightarrow$ D $\rightarrow$ H $\rightarrow$ A $\rightarrow$ F

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from arch import arch_model, arch_lm_test
6 from statsmodels.graphics.tsaplots import plot_acf
7
8 # STEP B: Data and returns
9 btc = yf.download("BTC-USD", start="2019-01-01",
10 end="2024-12-31")
11 returns = np.log(btc["Close"]).diff().dropna() * 100
12
13 # STEP C: Visual ARCH check
14 fig, axes = plt.subplots(1, 2, figsize=(14, 4))
15 axes[0].plot(returns**2, linewidth=0.6, color="orange")
16 axes[0].set_title("Squared Returns -- look for clusters")
17 plot_acf(returns**2, lags=30, ax=axes[1], zero=False)
18 axes[1].set_title("ACF of Squared Returns -- spikes = ARCH
19 effects")
20 plt.tight_layout(); plt.show()
21
22 # STEP E: Formal ARCH LM test
23 lm = arch_lm_test(returns, nlags=10)
24 print(f"ARCH LM p-value: {lm.pvalue:.6f}")
25 assert lm.pvalue < 0.05, "No ARCH effects -- GARCH not
26 needed"
27
28 # STEP G: Specify and fit GARCH(1,1)
29 model = arch_model(returns, mean="Constant", vol="GARCH",
30 p=1, q=1)

```

```

27 fit    = model.fit(dis="off")
28
29 # STEP D: Print summary
30 print(fit.summary())
31
32 # STEP H: Stationarity check
33 alpha = fit.params["alpha[1]"]; beta =
    fit.params["beta[1]"]
34 print(f"\nalpha + beta = {alpha+beta:.6f}  "
35       f"({'Stationary' if alpha+beta < 1 else 'IGARCH --
        non-stationary!'}))")
36
37 # STEP A: Standardised residuals -- check for remaining
    ARCH effects
38 lm_resid = arch_lm_test(fit.std_resid.dropna(), nlags=10)
39 print(f"ARCH LM on std resid: p={lm_resid.pvalue:.4f}  "
40       f"({'Clean' if lm_resid.pvalue > 0.05 else 'Still
        ARCH effects'}))")
41
42 # STEP F: Volatility forecast
43 fc = fit.forecast(horizon=22, reindex=False)
44 vol_fc = np.sqrt(fc.variance.iloc[0])
45 print(f"\n22-day vol forecast: {vol_fc.mean():.4f}% avg
        daily")

```

② Plain English

The GARCH workflow mirrors ARIMA's Box-Jenkins logic: visualise (C) → test formally (E) → fit (G) → inspect (D) → validate constraints (H) → check residuals (A) → forecast (F). Residual checking (Step A) is critical: if standardised residuals still show ARCH effects, GARCH(1,1) is insufficient.

③ Common Mistake

Skipping Step A (standardised residual check). Students often stop after Step D. But if z_t^2 still has significant ACF, the GARCH model hasn't captured all variance dynamics — you might need GARCH(2,1) or GJR-GARCH. The residual check is the GARCH equivalent of the Ljung-Box test for ARIMA.

④ Real World Link

In practice, Step A reveals that simple GARCH(1,1) rarely fully captures BTC's variance dynamics — standardised residuals often still show mild ARCH effects and heavy tails. This motivates the t-distribution GARCH or GJR-GARCH extensions. But GARCH(1,1) is still the starting point.

M4.014 Match the Following **MEDIUM**

Topic: GARCH model variants — applications

Match each **model** (Column A) to its **defining feature** (Column B).

#	Column A — Model	#	Column B — Defining Feature
1	ARCH(q)	(a)	Asymmetric — bad news raises vol more than good news
2	GARCH(1,1)	(b)	Only past q squared shocks — no lagged variance
3	GJR-GARCH	(c)	ARMA variance with 1 ARCH + 1 GARCH lag — industry default
4	EGARCH	(d)	Models $\log(\sigma^2)$ ensuring positivity without sign constraints
5	IGARCH	(e)	$\alpha + \beta = 1$ — volatility shocks are permanent

Solution

① Correct Matches

Model	Match
1. ARCH(q)	(b) Only past q squared shocks — no lagged variance
2. GARCH(1,1)	(c) ARMA variance with 1 ARCH + 1 GARCH lag — industry default
3. GJR-GARCH	(a) Asymmetric — bad news raises vol more than good news
4. EGARCH	(d) Models $\log(\sigma^2)$ — no sign constraints needed
5. IGARCH	(e) $\alpha + \beta = 1$ — permanent volatility shocks

② Plain English

The GARCH family tree: ARCH is the grandfather (simple but parameter-heavy). GARCH adds lagged variance for parsimony. GJR-GARCH adds an asymmetry term (the “leverage effect” — equity prices typically show higher vol after negative returns). EGARCH uses log-variance to avoid non-negativity constraints. IGARCH is the boundary case where shocks never dissipate.

③ Common Mistake

Confusing GJR-GARCH’s asymmetry with EGARCH’s. Both capture the leverage effect but differently: GJR-GARCH adds a separate term for negative shocks. EGARCH uses a different parametrisation entirely (log variance + signed shock terms). In the `arch` library: GJR-GARCH uses `o=1` parameter; EGARCH uses `vol="EGARCH"`.

④ Real World Link

For BTC, the leverage effect is reversed compared to equities — negative returns often create buying opportunities (“buy the dip” mentality) and can sometimes reduce volatility as panic sellers exhaust themselves. This “inverse leverage effect” means EGARCH may fit BTC better than GJR-GARCH. Testing multiple specifications and comparing AIC is essential.

M4.015 Multiple Choice **MEDIUM****Topic:** Impulse Response Function (IRF) interpretation

In a VAR model for BTC and Gold returns, the Impulse Response Function (IRF) shows that a 1-standard-deviation shock to BTC returns causes Gold returns to rise for 2 periods then return to zero. What does this indicate?

- (A) BTC and Gold have a permanent long-run equilibrium relationship (cointegration)
- (B) BTC shocks have a temporary (transitory) effect on Gold that dissipates after 2 periods
- (C) The VAR model is misspecified — IRFs should always be zero after 1 period
- (D) Gold Granger-causes BTC with a 2-period lag

Solution**① Correct Answer: (B)****② Plain English**

An IRF traces how a shock to one variable propagates through the VAR system over time. A BTC shock that elevates Gold returns for 2 periods then returns to zero is a transitory effect — the system is stationary and returns to equilibrium. This is expected for return series. Permanent effects (IRF doesn't return to zero) would indicate non-stationarity or cointegration — appropriate for price levels, not returns.

③ Common Mistake

Choosing (A) — cointegration is a concept for price LEVELS (not returns) and refers to a permanent long-run relationship detected by the Johansen test, not by IRF analysis. IRFs on stationary return series will always eventually return to zero for stable VAR systems.

④ Real World Link

If BTC shocks truly have a 2-day transitory effect on Gold, a trader could buy Gold immediately after a large BTC rally and sell 2 days later — a statistical arbitrage signal. The VAR/IRF framework is exactly how quantitative analysts search for such cross-asset lead-lag signals.

M4.016 Write from Scratch **MEDIUM****Topic:** Volatility clustering visualisation dashboard

Write code that creates a 4-panel volatility analysis dashboard for BTC showing: (1) log returns time series, (2) squared returns, (3) ACF of squared returns, (4) rolling 30-day annualised volatility.

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.graphics.tsaplots import plot_acf
6
7 btc      = yf.download("BTC-USD", start="2019-01-01",
8                       end="2024-12-31")
9 returns = np.log(btc["Close"]).diff().dropna() * 100
10
11 fig, axes = plt.subplots(4, 1, figsize=(16, 16))
12 fig.suptitle("BTC-USD -- Volatility Clustering Analysis
13             Dashboard",
14             fontsize=14, fontweight="bold", y=1.01)
15
16 # Panel 1: Log Returns
17 ax = axes[0]
18 ax.plot(returns, color="steelblue", linewidth=0.6,
19        alpha=0.8)
20 ax.axhline(0, color="black", linewidth=0.8)
21 ax.set_title("Panel 1: Daily Log Returns (%)")
22 ax.set_ylabel("Return (%)")
23 threshold = returns.std() * 2
24 ax.scatter(returns[returns > threshold].index,
25           returns[returns > threshold], color="green",
26           s=8,
27           zorder=5, label="Large gain")
28 ax.scatter(returns[returns < -threshold].index,
29           returns[returns < -threshold], color="red", s=8,
30           zorder=5, label="Large loss")
31 ax.legend(fontsize=8)
32
33 # Panel 2: Squared Returns (variance proxy)
34 ax = axes[1]
35 sq_returns = returns ** 2
36 ax.fill_between(sq_returns.index, sq_returns, 0,
37               color="orange", alpha=0.6)
38 ax.set_title("Panel 2: Squared Returns (r^2) -- Variance
39             Proxy\n"
40             "Clusters of high values = ARCH effects")
41 ax.set_ylabel("Squared Return (%^2)")
42
43 # Panel 3: ACF of Squared Returns
44 ax = axes[2]
45 plot_acf(sq_returns, lags=40, alpha=0.05, ax=ax,
46         zero=False,
47         color="orange")
48 ax.set_title("Panel 3: ACF of Squared Returns\n")

```

```

43         "Significant spikes = ARCH effects (variance
         autocorrelation)")
44 ax.set_ylabel("ACF")
45
46 # Panel 4: Rolling 30-Day Annualised Volatility
47 ax = axes[3]
48 roll_vol = returns.rolling(window=30).std() * np.sqrt(252)
49 ax.fill_between(roll_vol.index, roll_vol, 0,
50                 color="purple", alpha=0.4, label="Rolling
                    30-Day Vol")
51 ax.plot(roll_vol, color="purple", linewidth=1)
52 avg_vol = roll_vol.mean()
53 ax.axhline(avg_vol, color="red", linewidth=1.5,
54            linestyle="--",
55            label=f"Avg: {avg_vol:.1f}% p.a.")
56 ax.set_title("Panel 4: Rolling 30-Day Annualised
57              Volatility (%)\\n"
58              "Persistent spikes = volatility regime
              changes")
59 ax.set_ylabel("Annualised Vol (%)"); ax.legend(fontsize=9)
60
61 plt.tight_layout()
62 plt.savefig("btc_volatility_dashboard.png", dpi=120,
63             bbox_inches="tight")
64 plt.show()
65
66 print(f"\\nDashboard Summary:")
67 print(f"  Mean daily return   : {returns.mean():.4f}%")
68 print(f"  Daily return std     : {returns.std():.4f}%")
69 print(f"  Avg annualised vol    : {avg_vol:.2f}% p.a.")
70 print(f"  Max rolling 30d vol: {roll_vol.max():.2f}%
71        f" (during {roll_vol.idxmax().strftime('%b %Y')})")
72 print(f"  Skewness              : {returns.skew():.4f}")
73 print(f"  Excess kurtosis       : {returns.kurtosis():.4f}
74        (0 = normal)")

```

② Plain English

This dashboard provides visual evidence for all ARCH effect indicators simultaneously. Panel 1 shows clustering visually (stormy periods look dense). Panel 2 makes variance clustering explicit. Panel 3 gives statistical confirmation via ACF. Panel 4 summarises volatility regimes in percentage terms for investors and risk managers.

③ Common Mistake

Annualising daily volatility by multiplying by 252 instead of $\sqrt{252}$. Daily standard deviation $\times \sqrt{252}$ gives annual standard deviation. Multiplying by 252 gives annual variance, not volatility — producing numbers like 2000% that look absurdly large.

④ Real World Link

This exact 4-panel dashboard is used by crypto trading desks for daily monitoring. Panel 4 (rolling volatility) feeds directly into position sizing — many systematic funds halve their BTC position when rolling 30-day vol exceeds 100% annualised, and increase it when vol drops below 60%. The dashboard makes these regime changes actionable.

M4.017 Fill in the Blank **MEDIUM**

Topic: Granger causality test in a VAR framework

Fill in the blanks to run Granger causality tests between BTC, MRF, and Gold.

```

1 from statsmodels.tsa.vector_ar.var_model import VAR
2
3 btc = yf.download("BTC-USD", start="2020-01-01",
4   end="2024-12-31")
5 mrf = yf.download("MRF.NS", start="2020-01-01",
6   end="2024-12-31")
7 gold = yf.download("GC=F", start="2020-01-01",
8   end="2024-12-31")
9
10 data = pd.DataFrame({
11     "BTC" : np.log(btc["Close"]).diff(),
12     "MRF" : np.log(mrf["Close"]).diff(),
13     "Gold": np.log(gold["Close"]).diff()
14 })
15
16 fit = VAR(data).fit(maxlags=5, ic="____")
17
18 # Does BTC Granger-cause MRF?
19 gc_btc_mrf = fit.____(
20     caused = "____",
21     causing = ["____"]
22 )
23 print(f"BTC -> MRF: F={gc_btc_mrf.test_statistic:.4f}, "
24       f" p={gc_btc_mrf.____:.4f}")
25
26 # Does Gold Granger-cause BTC?
27 gc_gold_btc = fit.test_causality(caused="BTC",
28     causing=["____"])
29 print(f"Gold -> BTC: F={gc_gold_btc.test_statistic:.4f}, "
30       f" p={gc_gold_btc.pvalue:.4f}")

```

Solution

① Correct Code


```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import warnings; warnings.filterwarnings('ignore')
4 from statsmodels.tsa.vector_ar.var_model import VAR
5
6 btc = yf.download("BTC-USD", start="2020-01-01",
7 end="2024-12-31")
8 mrf = yf.download("MRF.NS", start="2020-01-01",
9 end="2024-12-31")
10 gold = yf.download("GC=F", start="2020-01-01",
11 end="2024-12-31")
12
13 data = pd.DataFrame({
14     "BTC" : np.log(btc["Close"]).diff(),
15     "MRF" : np.log(mrf["Close"]).diff(),
16     "Gold": np.log(gold["Close"]).diff()
17 }).dropna() # align dates and drop NaN
18
19 fit = VAR(data).fit(maxlags=5, ic="aic") # AIC lag
20 selection
21 print(f"VAR fitted: {fit.k_ar} lags, {fit.neqs} variables")
22
23 # Does BTC Granger-cause MRF?
24 gc_btc_mrf = fit.test_causality(
25     caused = "MRF", # variable being
26     predicted
27     causing = ["BTC"] # variable whose
28     lags we test
29 )
30 print(f"\nBTC -> MRF: F={gc_btc_mrf.test_statistic:.4f},"
31 f" p={gc_btc_mrf.pvalue:.4f}")
32 print(f" {'BTC Granger-causes MRF' if gc_btc_mrf.pvalue <
33 0.05 else 'No causality'}")
34
35 # Does Gold Granger-cause BTC?
36 gc_gold_btc = fit.test_causality(caused="BTC",
37 causing=["Gold"])
38 print(f"\nGold -> BTC: F={gc_gold_btc.test_statistic:.4f},"
39 f" p={gc_gold_btc.pvalue:.4f}")
40 print(f" {'Gold Granger-causes BTC' if gc_gold_btc.pvalue
41 < 0.05 else 'No causality'}")
42
43 # Full causality matrix
44 cols = list(data.columns)
45 print("\nFull Granger Causality Matrix (p-values) (* =
46 significant at 5%):")
47 print(f"{'':6}", end="")
48 for c in cols: print(f"{c:>10}", end="")
49 print()

```

```

40 for caused in cols:
41     print(f"{caused:6}", end=" ")
42     for causing in cols:
43         if caused == causing: print(f"'---':>10", end=" ")
44         else:
45             gc = fit.test_causality(caused=caused,
46                                   causing=[causing])
47             m = "*" if gc.pvalue < 0.05 else " "
48             print(f"{gc.pvalue:>9.4f}{m}", end=" ")
49     print()

```

② Plain English

`test_causality(caused="MRF", causing=["BTC"])` tests H_0 : BTC's lagged values have zero coefficients in MRF's VAR equation. The full causality matrix shows all pairwise relationships simultaneously. Significant p-values reveal which assets "lead" others in the VAR system.

③ Common Mistake

Reversing `caused` and `causing` arguments. "BTC Granger-causes MRF" means we test whether BTC's lags improve MRF forecasts — so `caused="MRF"` and `causing=["BTC"]`. Always read: "Does [causing] Granger-cause [caused]?"

④ Real World Link

The BTC→MRF Granger causality is plausible through the FII sentiment channel — large BTC moves affect global risk sentiment, which flows into FII selling or buying of Indian equities including MRF. If detected, this creates a cross-asset signal: a large BTC decline today might predict a slight MRF decline tomorrow.

M4.018 Find the Error MEDIUM

Topic: GJR-GARCH — leverage effect interpretation

This code correctly fits a GJR-GARCH but incorrectly interprets the leverage coefficient. Find the interpretation error.

```

1 from arch import arch_model
2
3 btc      = yf.download("BTC-USD", start="2019-01-01",
4                       end="2024-12-31")
5 returns = np.log(btc["Close"]).diff().dropna() * 100
6
7 model = arch_model(returns, mean="Constant", vol="GARCH",
8                   p=1, o=1, q=1)
9 fit    = model.fit(dispatch="off")
10
11 gamma = fit.params["gamma[1]"]
12 print(f"Gamma (leverage): {gamma:.4f}")
13
14 # WRONG INTERPRETATION:
15 if gamma > 0:

```

```

14 print("Positive gamma -> BTC rises MORE after positive
    shocks") # WRONG
15 print("Good news causes more volatility than bad news
    for BTC") # WRONG

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import warnings; warnings.filterwarnings('ignore')
4 from arch import arch_model
5
6 btc      = yf.download("BTC-USD", start="2019-01-01",
    end="2024-12-31")
7 returns = np.log(btc["Close"]).diff().dropna() * 100
8
9 model = arch_model(returns, mean="Constant", vol="GARCH",
    p=1, o=1, q=1)
10 fit    = model.fit(dis="off")
11 print(fit.summary())
12
13 alpha = fit.params["alpha[1]"]
14 gamma = fit.params["gamma[1]"]
15
16 print(f"\nAlpha (symmetric ARCH): {alpha:.4f}")
17 print(f"Gamma (leverage term) : {gamma:.4f}")
18
19 # CORRECT INTERPRETATION:
20 # GJR-GARCH:  $\sigma_t^2 = \omega + (\alpha + \gamma I_{t-1})\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$ 
21 #  $I_{t-1} = 1$  if  $\epsilon_{t-1} < 0$  (negative shock indicator)
22 #
23 # POSITIVE gamma -> NEGATIVE shocks raise vol MORE than
    positive shocks
24 # Effect of positive shock:  $\alpha * \epsilon^2$ 
25 # Effect of negative shock:  $(\alpha + \gamma) * \epsilon^2$  ->
    larger if  $\gamma > 0$ 
26 #
27 # For BTC specifically, gamma may be NEGATIVE (inverse
    leverage):
28 # Positive shocks (FOMO rallies) can increase BTC
    volatility MORE
29 # This is the OPPOSITE of equity markets
30
31 if gamma > 0:

```

```

32     print("\nPositive gamma -> NEGATIVE BTC shocks cause
        MORE volatility")
33     print("    (Classical leverage effect -- equity-like
        asymmetry)")
34 elif gamma < 0:
35     print("\nNegative gamma -> POSITIVE BTC shocks cause
        MORE volatility")
36     print("    (Inverse leverage effect -- FOMO rallies are
        more volatile)")
37     print("    This is common for cryptocurrencies and
        commodity markets")
38 else:
39     print("\nGamma ~= 0: Symmetric volatility response")

```

② Plain English

In GJR-GARCH, the I_{t-1} indicator equals 1 when the previous return was negative. So positive gamma means negative-return days contribute $(\alpha + \gamma)$ to tomorrow's variance, while positive-return days contribute only α . The original code stated “positive gamma $\rightarrow$ volatility rises MORE after positive shocks” — precisely backwards.

③ Common Mistake

Getting the sign interpretation wrong leads to completely inverted conclusions about market asymmetry. Positive gamma in GJR-GARCH amplifies the variance effect of NEGATIVE shocks (when $I_{t-1} = 1$, i.e., when $\varepsilon_{t-1} < 0$). Always map the indicator function carefully.

④ Real World Link

The sign and magnitude of γ for BTC is an active research question. During BTC bull markets, upward price rallies are often accompanied by higher volatility (retail FOMO, leverage unwinding at tops). During bear markets, declines can be orderly. This suggests $\gamma < 0$ for BTC — an “inverse leverage effect” — contrasting with equities like MRF ($\gamma > 0$, classical leverage effect).

M4.019 Multiple Choice MEDIUM

Topic: VAR — number of parameters

A VAR(2) model is fitted to a 3-variable system (BTC, MRF, Gold returns) with an intercept in each equation. How many parameters does the model have in total?

- (A) 6 — one coefficient per variable per equation
- (B) 18 — 2 lags $\times$ 3 variables $\times$ 3 equations, no intercepts
- (C) 21 — 2 lags $\times$ 3 variables $\times$ 3 equations + 3 intercepts (one per equation)
- (D) 9 — 3 variables $\times$ 3 coefficients, regardless of lags

Solution**① Correct Answer: (C)****② Plain English**

Each VAR equation models one variable using its own lags AND the lags of all other variables. With $k = 3$ variables and $p = 2$ lags, each equation has $k \times p = 3 \times 2 = 6$ slope coefficients plus 1 intercept = 7 parameters. With 3 equations: $3 \times 7 = 21$ parameters total. This rapid parameter growth with k and p is why VAR models are only practical for small numbers of variables.

③ Common Mistake

Forgetting to count the intercept in each equation. Many students compute $2 \times 3 \times 3 = 18$ and stop. Each of the 3 equations also has a constant term, adding 3 more parameters. With $n = 1000$ trading-day observations and 21 parameters, you have roughly 48 observations per parameter — borderline adequate.

④ Real World Link

The “curse of dimensionality” is the central limitation of VAR models. VAR is typically used with very few variables (2–4) or at lower frequencies (weekly, monthly) where the parameter-to-observation ratio is more favourable. Larger systems require penalised VAR methods like LASSO-VAR.

M4.020 Write from Scratch MEDIUM

Topic: Value-at-Risk (VaR) from GARCH

Write code that: (1) fits GARCH(1,1) to BTC returns, (2) generates 1-day ahead 99% VaR using GARCH conditional volatility, (3) backtests the VaR — counts how many times actual losses exceeded VaR, (4) computes the VaR violation rate and compares to expected 1%, (5) plots actual returns vs VaR threshold over the last 500 days.

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as stats
4 import warnings; warnings.filterwarnings('ignore')
5 from arch import arch_model
6
7 btc      = yf.download("BTC-USD", start="2019-01-01",
8                       end="2024-12-31")
9 returns = np.log(btc["Close"]).diff().dropna() * 100
10
11 model = arch_model(returns, mean="Constant", vol="GARCH",
12                   p=1, q=1)
13 fit    = model.fit(dispatch="off")

```

```

12
13 # 1-Day 99% VaR:  $VaR = \mu + z_{\{0.01\}} \times \sigma_t$ 
14 mu = fit.params["mu"]
15 cond_vol = fit.conditional_volatility
16 z_99 = stats.norm.ppf(0.01) # -2.326
17
18 var_99 = mu + z_99 * cond_vol # negative value =
    potential loss
19
20 # Backtest
21 aligned = pd.DataFrame({"return": returns, "VaR_99":
    var_99}).dropna()
22 violations = aligned["return"] < aligned["VaR_99"]
23 n_viol = violations.sum()
24 viol_rate = violations.mean()
25
26 print("="*52)
27 print(" GARCH(1,1) VaR Backtest -- BTC-USD")
28 print("="*52)
29 print(f" Evaluation period : {len(aligned)} trading
    days")
30 print(f" Expected VaR breaks: {len(aligned)*0.01:.1f}
    (1%)")
31 print(f" Actual VaR breaks : {n_viol}")
32 print(f" Violation rate : {viol_rate*100:.2f}%")
33 if abs(viol_rate - 0.01) < 0.005:
34     print(" Model well-calibrated (within 0.5pp of
        target)")
35 elif viol_rate > 0.015:
36     print(" Too many violations -- VaR underestimates
        risk")
37 else:
38     print(" VaR over-conservative -- overestimates risk")
39 print("="*52)
40
41 # Plot last 500 days
42 plot_data = aligned.tail(500)
43 fig, ax = plt.subplots(figsize=(16, 6))
44 ax.plot(plot_data.index, plot_data["return"],
    color="steelblue", linewidth=0.7, alpha=0.8,
    label="BTC Daily Return (%)")
45 ax.plot(plot_data.index, plot_data["VaR_99"],
    color="red", linewidth=1.5, label="99% VaR (loss
    threshold)")
46 ax.fill_between(plot_data.index, plot_data["return"],
    plot_data["VaR_99"],
47     where=(plot_data["return"] <
        plot_data["VaR_99"]),

```

```

50         color="red", alpha=0.4, label="VaR
           Violation")
51 ax.axhline(0, color="black", linewidth=0.8)
52 ax.set_title("BTC-USD -- GARCH(1,1) 99% VaR Backtest (Last
           500 Days)")
53 ax.set_ylabel("Return / VaR (%)"); ax.legend(fontsize=9)
54 plt.tight_layout(); plt.show()
55
56 print(f"\nAverage 99% VaR : {var_99.mean():.4f}%
           (potential loss per day)")
57 print(f"Worst VaR day      : {var_99.min():.4f}% on
           {var_99.idxmin().strftime('%Y-%m-%d')}")

```

② Plain English

VaR answers: “On a bad day (worst 1% of days), how much will I lose?” GARCH gives a time-varying VaR — tighter on calm days, wider during volatile regimes. The backtest validates calibration: if 1.5% of days show losses worse than the 99% VaR (expected: 1%), the model underestimates risk.

③ Common Mistake

Forgetting that VaR is a loss threshold — it should be a negative number. $z_{99} = \text{stats.norm.ppf}(0.01) = -2.326$, so $\text{VaR} = \mu + (-2.326) \times \sigma$ will be negative for reasonable parameters. A positive VaR would mean you expect to gain money on bad days — nonsensical.

④ Real World Link

Every regulated financial institution holding BTC must report VaR. Basel III requires banks to backtest their VaR models — if violations exceed 10 in 250 trading days (4%), the model is considered inadequate and capital requirements are increased. GARCH-based VaR is the industry standard because it adapts to volatility regimes.

M4.021 Fill in the Blank MEDIUM

Topic: VAR forecasting and Forecast Error Variance Decomposition (FEVD)
Fill in the blanks to generate VAR forecasts and compute FEVD for the BTC-Gold system.

```

1 from statsmodels.tsa.vector_ar.var_model import VAR
2
3 btc  = yf.download("BTC-USD", start="2020-01-01",
4                   end="2024-12-31")
5
6 gold = yf.download("GC=F",      start="2020-01-01",
7                   end="2024-12-31")
8
9 data = pd.DataFrame({
10     "BTC" : np.log(btc["Close"]).diff(),
11     "Gold": np.log(gold["Close"]).diff()
12 })

```

```

10
11 fit = VAR(data).fit(maxlags=5, ic="aic")
12
13 # 10-step ahead forecast
14 fc = fit._____(steps=10, y=data.values[-fit.k_ar:])
15
16 print("10-step VAR Forecast:")
17 print(pd.DataFrame(fc, columns=data.______))
18
19 # Forecast Error Variance Decomposition (FEVD)
20 fevd = fit._____(10)          # FEVD method
21 fevd._____( )                # plot method
22 plt.suptitle("FEVD: BTC-Gold System", fontsize=12)
23 plt.tight_layout(); plt.show()
24
25 # FEVD table at horizon 5
26 print("\nFEVD at horizon 5:")
27 print(fevd._____(5))          # decomp method at horizon h

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.vector_ar.var_model import VAR
6
7 btc = yf.download("BTC-USD", start="2020-01-01",
8   end="2024-12-31")
9 gold = yf.download("GC=F", start="2020-01-01",
10   end="2024-12-31")
11
12 data = pd.DataFrame({
13     "BTC" : np.log(btc["Close"]).diff(),
14     "Gold": np.log(gold["Close"]).diff()
15 }).dropna().loc[lamba df: df.index.isin(
16     btc["Close"].dropna().index &
17     gold["Close"].dropna().index)]
18 data = data.dropna()
19
20 fit = VAR(data).fit(maxlags=5, ic="aic")
21 print(f"VAR({fit.k_ar}) | AIC={fit.aic:.2f}")
22
23 # 10-step ahead forecast (needs last k_ar obs as seed)
24 fc = fit.forecast(y=data.values[-fit.k_ar:], steps=10)
25

```



```

23 print("\n10-step VAR Forecast (log returns):")
24 print(pd.DataFrame(fc, columns=data.columns))
25
26 # FEVD
27 fevd = fit.fevd(10)
28 fevd.plot()
29 plt.suptitle("FEVD: What drives BTC and Gold forecast
    errors?", fontsize=11)
30 plt.tight_layout(); plt.show()
31
32 # FEVD at horizon 5 (index 4 = 0-indexed)
33 print("\nFEVD at horizon 5:")
34 fevd_5 = pd.DataFrame(fevd.decomp[4],
35                       index=data.columns,
36                       columns=data.columns)
37
38 print(fevd_5.round(4))
39 print(f"\n {fevd_5.loc['BTC','BTC']*100:.1f}% of BTC
    forecast error = BTC's own shocks")
40 print(f" {fevd_5.loc['BTC','Gold']*100:.1f}% of BTC
    forecast error = Gold shocks")
41 print(f" {fevd_5.loc['Gold','Gold']*100:.1f}% of Gold
    forecast error = Gold's own shocks")

```

② Plain English

FEVD answers: “Of all the uncertainty in my forecast for BTC returns at horizon h , what fraction comes from BTC’s own unpredictable shocks vs Gold’s shocks?” If Gold explains 0.5% of BTC’s forecast error variance and BTC explains 99.5%, then Gold is basically irrelevant to BTC’s future.

③ Common Mistake

Confusing FEVD with Granger causality. Granger causality tests whether including the other variable’s lags significantly improves in-sample fit. FEVD shows the fraction of forecast error variance attributable to each variable’s structural shocks (after orthogonalisation). A variable can Granger-cause another but explain very little FEVD if its effect is small in magnitude.

④ Real World Link

FEVD is used in macroeconomics to understand what fraction of GDP forecast uncertainty comes from monetary policy shocks vs oil price shocks. For BTC-Gold, it quantifies whether “gold is a safe haven for crypto holders” — if large fractions of BTC forecast variance come from Gold shocks, they’re tightly linked.

M4.022 Find the Error MEDIUM

Topic: GARCH — Student distribution and degrees of freedom

This code fits a GARCH model with Student-t innovations but extracts the wrong parameter name for degrees of freedom.

```

1 from arch import arch_model

```

```

2
3 btc      = yf.download("BTC-USD", start="2019-01-01",
4                       end="2024-12-31")
5 returns = np.log(btc["Close"]).diff().dropna() * 100
6
7 model = arch_model(returns, mean="Constant", vol="GARCH",
8                   p=1, q=1,
9                   dist="t")
10 fit    = model.fit(dispatch="off")
11
12 print(fit.params)
13
14 # ERROR: trying to access non-existent parameter name
15 nu = fit.params["degrees_of_freedom"] # ERROR: wrong
    parameter name
16 print(f"Degrees of freedom: {nu:.2f}")
17 print(f"Tail heaviness: {'Very heavy (nu<5)' if nu < 5
18       else 'Moderate'}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import warnings; warnings.filterwarnings('ignore')
4 from arch import arch_model
5
6 btc      = yf.download("BTC-USD", start="2019-01-01",
7                       end="2024-12-31")
8 returns = np.log(btc["Close"]).diff().dropna() * 100
9
10 model = arch_model(returns, mean="Constant", vol="GARCH",
11                   p=1, q=1,
12                   dist="t")
13 fit    = model.fit(dispatch="off")
14
15 # FIX: Print params first to find the actual parameter name
16 print("All estimated parameters:")
17 print(fit.params)
18 print()
19
20 # In arch library, degrees of freedom parameter is named
21   "nu" (Greek letter nu)
22 nu = fit.params["nu"] # FIX: correct name is "nu"
23 print(f"Degrees of freedom (nu): {nu:.4f}")
24
25 # Interpretation of nu:

```

```

23 # nu -> inf : t-distribution converges to Normal (thin
    tails)
24 # nu = 30    : Close to Normal -- mild fat tails
25 # nu = 10    : Moderate fat tails
26 # nu = 5     : Heavy fat tails (4th moment barely exists)
27 # nu <= 4    : Variance of distribution is infinite
28
29 if nu < 5:
30     print("Very heavy tails -- 4th moment may be
        undefined")
31     print("Common for BTC -- extreme returns much more
        frequent than Normal predicts")
32 elif nu < 10:
33     print("Moderate-to-heavy tails")
34 else:
35     print("Near-Normal tails at this nu")
36
37 # Compare Normal vs t-GARCH by AIC
38 model_n = arch_model(returns, mean="Constant",
    vol="GARCH", p=1, q=1,
39                     dist="normal")
40 fit_n    = model_n.fit(dispatch="off")
41
42 print(f"\nAIC comparison:")
43 print(f"    GARCH(1,1) Normal : {fit_n.aic:.2f}")
44 print(f"    GARCH(1,1) t-dist : {fit.aic:.2f}")
45 print(f"    t-dist better: {fit.aic < fit_n.aic}")

```

② Plain English

When using `dist="t"` in the `arch` library, the degrees-of-freedom parameter is stored as `"nu"` (the Greek letter ν). Printing `fit.params` first is always the safest way to discover exact parameter names — never guess. The AIC comparison typically shows that t-distribution GARCH significantly outperforms Normal GARCH for BTC ($\nu \approx 4\text{--}7$ is typical, indicating substantial fat tails).

③ Common Mistake

Using `fit.params["df"]`, `fit.params["dof"]`, or `fit.params["degrees_of_freedom"]` — none of these are correct. The `arch` library uses Greek letter names for distribution parameters: `"nu"` for t-degrees of freedom. Always run `print(fit.params)` and read the output before trying to access specific parameters.

④ Real World Link

BTC's degrees-of-freedom parameter $\nu \approx 4\text{--}6$ in a t-GARCH model means the distribution has extreme kurtosis. This explains why “100-year events” in BTC (like the FTX collapse dropping 25% in two days) occur every few years rather than once per century. Normal-distribution VaR would catastrophically underestimate these tail risks.

Section C

Hard Problems — M4.023 to M4.034

M4.023 Fill in the Blank **HARD**

Topic: EGARCH — log-variance model

Fill in the blanks to fit an EGARCH model to Gold returns and compare its leverage structure with GARCH(1,1).

```

1 from arch import arch_model
2
3 gold      = yf.download("GC=F", start="2019-01-01",
4                       end="2024-12-31")
5 returns = np.log(gold["Close"]).diff().dropna() * 100
6
7 # EGARCH(1,1) -- models log(sigma^2) directly
8 model_egarch = arch_model(
9     returns,
10    mean = "Constant",
11    vol  = "_____", # EGARCH
12        specification
13    p    = 1,
14    q    = 1
15)
16 fit_eg = model_egarch.fit(dispatch="off")
17 fit_g   = arch_model(returns, mean="Constant",
18                     vol="GARCH", p=1, q=1).fit(dispatch="off")
19
20 print("EGARCH Parameters:"); print(fit_eg.params)
21
22 # EGARCH: log(sigma^2_t) = omega + alpha*(|z_{t-1}| -
23           E|z|) + gamma*z_{t-1} + beta*log(sigma^2_{t-1})
24 # gamma < 0 -> negative shocks raise vol more (leverage
25           effect, equity-like)
26 # gamma > 0 -> positive shocks raise vol more (inverse
27           leverage, crypto-like)
28 gamma_eg = fit_eg.params["_____"]
29
30 print(f"\nEGARCH gamma (asymmetry): {gamma_eg:.4f}")
31 if gamma_eg < 0:
32     print("Classical leverage: bad news -> more vol")
33 elif gamma_eg > 0:
34     print("Inverse leverage: good news -> more vol")
35
36 print(f"\nAIC GARCH : {fit_g.aic:.2f}")
37 print(f"AIC EGARCH : {fit_eg._____.2f}")
38 print(f"EGARCH wins: {fit_eg.aic < fit_g._____}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from arch import arch_model
6
7 gold = yf.download("GC=F", start="2019-01-01",
8 end="2024-12-31")
9 returns = np.log(gold["Close"]).diff().dropna() * 100
10 model_egarch = arch_model(returns, mean="Constant",
11 vol="EGARCH", p=1, q=1) #
12                               EGARCH specification
13 fit_eg = model_egarch.fit(dispatch="off")
14 fit_g = arch_model(returns, mean="Constant",
15 vol="GARCH", p=1, q=1).fit(dispatch="off")
16 print("EGARCH Parameters:"); print(fit_eg.params); print()
17
18 # In arch library, EGARCH asymmetry parameter is named
19 # "gamma[1]"
20 gamma_eg = fit_eg.params["gamma[1]"]
21 print(f"EGARCH gamma (asymmetry): {gamma_eg:.4f}")
22 if gamma_eg < 0:
23     print("Classical leverage: negative shocks -> more
24     volatility")
25     print("Gold behaves like equities -- fear dominates
26     greed")
27 elif gamma_eg > 0:
28     print("Inverse leverage: positive shocks -> more
29     volatility")
30     print("Gold rallies are more chaotic than Gold
31     declines")
32 else:
33     print("Symmetric: positive and negative shocks ->
34     equal vol impact")
35
36 print(f"\nModel Comparison:")
37 print(f"AIC GARCH(1,1): {fit_g.aic:.2f}")
38 print(f"AIC EGARCH(1,1): {fit_eg.aic:.2f}") # .aic
39                                         attribute
40 print(f"EGARCH wins by AIC: {fit_eg.aic < fit_g.aic}")
41
42 # Compare conditional volatility
43 fig, ax = plt.subplots(figsize=(14, 5))
44 ax.plot(fit_g.conditional_volatility, label="GARCH(1,1)",
45 linewidth=1)

```

```

38 ax.plot(fit_eg.conditional_volatility,
        label="EGARCH(1,1)", linewidth=1,
39         linestyle="--", color="red")
40 ax.set_title("Gold Futures -- GARCH vs EGARCH Conditional
        Volatility")
41 ax.set_ylabel("Daily Vol (%)"); ax.legend();
    plt.tight_layout(); plt.show()

```

② Plain English

EGARCH models $\log(\sigma_t^2)$ instead of σ_t^2 directly. This has two advantages: (1) log-variance is unconstrained, so all parameters can take any sign without non-negativity restrictions; (2) the γ parameter directly captures asymmetry — positive γ means positive return shocks increase log-variance more. For Gold, the sign of γ reveals whether the market is more volatile during rallies (safe-haven buying panic) or selloffs (liquidation panic).

③ Common Mistake

Confusing EGARCH's γ interpretation with GJR-GARCH's γ . In GJR-GARCH, positive γ means negative shocks have extra impact. In EGARCH, positive γ means positive standardised residuals increase log-variance more. The signs have opposite conventional meanings — always check the library documentation.

④ Real World Link

Gold's EGARCH asymmetry tells a macroeconomic story: during geopolitical crises (Russia-Ukraine 2022, Middle East tensions 2023), Gold rallies rapidly and volatility surges — a positive asymmetry signature. Understanding this asymmetry helps options traders price Gold calls vs puts correctly, as the two directions of movement have different volatility dynamics.

M4.024 Find the Error **HARD**

Topic: VAR impulse response — wrong orthogonalisation

This code computes IRFs but the results are uninterpretable because of a critical methodological error. Find and explain it.

```

1 from statsmodels.tsa.vector_ar.var_model import VAR
2
3 btc = yf.download("BTC-USD", start="2020-01-01",
4                  end="2024-12-31")
5
6 gold = yf.download("GC=F", start="2020-01-01",
7                  end="2024-12-31")
8
9 data = pd.DataFrame({
10     "BTC" : np.log(btc["Close"]).diff(),
11     "Gold": np.log(gold["Close"]).diff()
12 }).dropna()
13
14 fit = VAR(data).fit(maxlags=5, ic="aic")

```

```

13 # ERROR: using non-orthogonalised IRF on correlated system
14 irf = fit.irf( periods=10)
15 irf.plot()    # no orth=True -> correlated shocks
                # contaminate each variable's IRF
16 plt.show()
17 # If BTC and Gold innovations are correlated, a "pure BTC
    # shock" also
18 # partially represents a Gold shock simultaneously -- IRFs
    # uninterpretable

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.vector_ar.var_model import VAR
6
7 btc  = yf.download("BTC-USD", start="2020-01-01",
8                   end="2024-12-31")
9
10 gold = yf.download("GC=F",      start="2020-01-01",
11                   end="2024-12-31")
12
13 data = pd.DataFrame({
14     "BTC" : np.log(btc["Close"]).diff(),
15     "Gold": np.log(gold["Close"]).diff()
16 }).dropna().loc[lamba df: df.index.isin(
17     btc["Close"].dropna().index &
18     gold["Close"].dropna().index)]
19 data = data.dropna()
20
21 fit = VAR(data).fit(maxlags=5, ic="aic")
22
23 # Check contemporaneous correlation of residuals
24 resid_corr = pd.DataFrame(fit.resid).corr()
25 print("Residual correlation matrix:")
26 print(resid_corr.round(4))
27
28 # EXPLANATION:
29 # Non-orthogonalised IRFs trace the response to a unit
    # shock in the
30 # reduced-form residuals eps_t. But if BTC and Gold
    # residuals are correlated
31 # (rho != 0), a shock to eps_BTC always coincides with a
    # partial shock to

```

```

28 # eps_Gold simultaneously. The IRF does NOT show the
    effect of a "pure BTC
29 # shock" in isolation.
30 #
31 # FIX: Use orthogonalised IRF (orth=True) -- applies
    Cholesky decomposition
32 # to transform correlated residuals into uncorrelated
    structural shocks.
33 # This allows interpreting "what happens to Gold when ONLY
    BTC shocks?"
34
35 irf = fit.irf( periods=10 )
36
37 # CORRECT: Orthogonalised IRF with Cholesky decomposition
38 irf.plot(orth=True, plot_stderr=True)
39 plt.suptitle("Orthogonalised IRF (Cholesky -- structurally
    interpretable)",
40             fontsize=10)
41 plt.tight_layout(); plt.show()
42
43 print("\nNote: Cholesky ordering matters -- BTC first
    means BTC shock is")
44 print("'pure'; Gold shock is what remains after removing
    BTC's contemporaneous effect.")
45 print("Different orderings give different orthogonalised
    IRFs -- check robustness!")

```

② Plain English

VAR residuals from different equations are almost always correlated — a BTC shock and a Gold shock tend to happen together (common macro news). Without orthogonalisation, IRF traces the response to a raw shock that contains information from multiple variables simultaneously, making it impossible to interpret as the effect of “just BTC” or “just Gold.” Cholesky orthogonalisation creates uncorrelated structural shocks by ordering variables and recursively partialing out correlations.

③ Common Mistake

Not checking whether residual correlation is high before interpreting IRFs. If BTC-Gold residual correlation is only 0.05 (near zero), non-orthogonalised and orthogonalised IRFs are nearly identical — the issue is minor. But if correlation is 0.4+, the two approaches give dramatically different results. Always report the residual correlation matrix.

④ Real World Link

The Cholesky ordering embeds an economic assumption — the first variable is assumed to respond to contemporaneous shocks in the second variable, but not vice versa within the same period. Whether this reflects actual market microstructure is an empirical question that requires careful justification. Report sensitivity to ordering choices in any academic submission.

M4.025 Multiple Choice HARD**Topic:** GARCH — long-run (unconditional) variance

In a GARCH(1,1) model with $\omega = 0.02$, $\alpha = 0.08$, $\beta = 0.90$ fitted to BTC daily percentage returns, what is the long-run (unconditional) variance, and what does it imply for long-run daily volatility?

- (A) Long-run variance = $0.02/(1 - 0.08 - 0.90) = 1.0$, daily vol = 1.0%
- (B) Long-run variance = $0.02/(1 - 0.90) = 0.2$, daily vol = $\sqrt{0.2} \approx 0.45\%$
- (C) Long-run variance = $\omega + \alpha + \beta = 1.0$, daily vol = 1.0%
- (D) Long-run variance = $\omega/(1 - \alpha - \beta) = 0.02/0.02 = 1.0$, daily vol = $\sqrt{1.0} = 1.0\%$

Solution**① Correct Answer: (D) — same result as (A) but correctly derived**

```

1  omega = 0.02; alpha = 0.08; beta = 0.90
2
3  # Long-run (unconditional) variance formula:
4  # sigma^2 = omega / (1 - alpha - beta)
5  # This is the GARCH equilibrium variance -- where
   sigma^2_t mean-reverts
6
7  long_run_var = omega / (1 - alpha - beta)
8  long_run_vol = long_run_var ** 0.5
9
10 print(f"1 - alpha - beta = {1 - alpha - beta:.2f}")
11 print(f"Long-run sigma^2 = {omega}/{1-alpha-beta:.2f} =
   {long_run_var:.4f}")
12 print(f"Long-run sigma    = sqrt({long_run_var:.4f}) =
   {long_run_vol:.4f}% per day")
13 print(f"Annualised sigma = {long_run_vol *
   (252**0.5):.2f}% per year")

```

② Plain English

The unconditional (long-run) variance $\omega/(1 - \alpha - \beta)$ is the level around which σ_t^2 fluctuates over time. It's the GARCH process's "equilibrium variance." The daily volatility of 1.0% corresponds to roughly 15.9% annualised, which is quite calm for BTC (actual long-run BTC vol is 3–4% daily $\approx$ 50–65% annualised).

③ Common Mistake

Choosing (B) — using $(1 - \beta) = 0.10$ in the denominator instead of $(1 - \alpha - \beta) = 0.02$. Both α and β together determine persistence. The correct denominator is always $(1 - \alpha - \beta)$.

④ Real World Link

For BTC with $\omega \approx 0.1$, $\alpha \approx 0.08$, $\beta \approx 0.90$, the long-run variance is $0.1/0.02 = 5$, daily vol $\approx 2.2\%$, annualised $\approx 35\%$. During volatile regimes ($\sigma_t \gg 2.2\%$), the GARCH process mean-reverts back toward 2.2% — the model "knows" that extreme volatility is temporary.

M4.026 Write from Scratch **HARD****Topic:** Multivariate GARCH — DCC (Dynamic Conditional Correlation)

Write code implementing Dynamic Conditional Correlation between BTC and Gold:
 (1) fit univariate GARCH(1,1) to each asset's returns, (2) compute standardised residuals, (3) estimate DCC correlation dynamics (EWMA approximation), (4) plot time-varying correlation vs static correlation, (5) identify periods when BTC-Gold correlation spiked.

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from arch import arch_model
6
7 btc = yf.download("BTC-USD", start="2019-01-01",
8   end="2024-12-31")
9 gold = yf.download("GC=F", start="2019-01-01",
10  end="2024-12-31")
11
12 btc_ret = np.log(btc["Close"]).diff().dropna() * 100
13 gold_ret = np.log(gold["Close"]).diff().dropna() * 100
14
15 combined = btc_ret.to_frame("BTC").join(
16   gold_ret.to_frame("Gold"),
17   how="inner").dropna()
18
19 # Step 1 & 2: Fit univariate GARCH, extract standardised
20 residuals
21 def fit_garch_std(returns, name):
22     model = arch_model(returns, mean="Constant",
23       vol="GARCH", p=1, q=1)
24     fit = model.fit(dis="off")
25     print(f"{name}: alpha={fit.params['alpha[1]']:.4f}, "
26       f"beta={fit.params['beta[1]']:.4f}")
27     return fit.std_resid.dropna()
28
29 z_btc = fit_garch_std(combined["BTC"], "BTC")
30 z_gold = fit_garch_std(combined["Gold"], "Gold")
31
32 z_df = z_btc.to_frame("z_BTC").join(
33   z_gold.to_frame("z_Gold"), how="inner").dropna()
34
35 # Step 3: DCC -- EWMA correlation approximation
36 lambda_dcc = 0.94 # RiskMetrics standard decay parameter
37 n = len(z_df)

```

```

33
34 q11 = np.var(z_df["z_BTC"]); q22 = np.var(z_df["z_Gold"])
35 q12 = np.cov(z_df["z_BTC"], z_df["z_Gold"])[0, 1]
36
37 Q11 = np.zeros(n); Q22 = np.zeros(n)
38 Q12 = np.zeros(n); rho = np.zeros(n)
39 Q11[0] = q11; Q22[0] = q22; Q12[0] = q12
40 rho[0] = q12 / np.sqrt(q11 * q22)
41
42 for t in range(1, n):
43     zb = z_df["z_BTC"].iloc[t-1]
44     zg = z_df["z_Gold"].iloc[t-1]
45     Q11[t] = lambda_dcc*Q11[t-1] + (1-lambda_dcc)*zb**2
46     Q22[t] = lambda_dcc*Q22[t-1] + (1-lambda_dcc)*zg**2
47     Q12[t] = lambda_dcc*Q12[t-1] + (1-lambda_dcc)*zb*zg
48     rho[t] = Q12[t] / np.sqrt(Q11[t] * Q22[t])
49
50 rho_series = pd.Series(rho, index=z_df.index)
51 static_corr = z_df.corr().loc["z_BTC", "z_Gold"]
52
53 # Step 4: Plot
54 fig, axes = plt.subplots(2, 1, figsize=(16, 10))
55
56 axes[0].plot(rho_series, color="purple", linewidth=1.2,
57             label="DCC (EWMA)")
58 axes[0].axhline(static_corr, color="red", linewidth=1.5,
59                 linestyle="--",
60                 label=f"Static rho = {static_corr:.4f}")
61 axes[0].axhline(0, color="black", linewidth=0.8)
62 axes[0].fill_between(rho_series.index, rho_series, 0,
63                     where=(rho_series > 0.1), alpha=0.2,
64                     color="green",
65                     label="High positive correlation (>0.1)")
66 axes[0].fill_between(rho_series.index, rho_series, 0,
67                     where=(rho_series < -0.1), alpha=0.2,
68                     color="red",
69                     label="High negative correlation (<-0.1)")
70 axes[0].set_title("BTC-Gold Dynamic Conditional Correlation (EWMA-DCC)")
71 axes[0].set_ylabel("Correlation rho_t");
72 axes[0].legend(fontsize=8)
73
74 # Step 5: Annotate top correlation periods
75 top5 = rho_series.nlargest(5).index
76 for d in top5:
77     axes[0].axvline(d, color="gray", linewidth=0.8,
78                     linestyle=":", alpha=0.6)

```

```

73     axes[0].annotate(d.strftime("%b\n%Y"), xy=(d,
74         rho_series[d]),
75         fontsize=7, color="darkgreen",
76         xytext=(3, 5),
77         textcoords="offset points")
78
79 axes[1].plot(combined["BTC"], color="orange",
80             linewidth=0.5, alpha=0.7,
81             label="BTC Return (%)")
82 axes[1].plot(combined["Gold"], color="gold",
83             linewidth=0.8, alpha=0.9,
84             label="Gold Return (%)")
85 axes[1].axhline(0, color="black", linewidth=0.8)
86 axes[1].set_title("BTC and Gold Daily Returns (context)");
87 axes[1].legend(fontsize=8)
88 plt.tight_layout(); plt.show()
89
90 print(f"\nDCC Correlation Summary:")
91 print(f"    Static correlation: {static_corr:.4f}")
92 print(f"    Mean DCC corr      : {rho_series.mean():.4f}")
93 print(f"    Max DCC corr       : {rho_series.max():.4f}
94       ({rho_series.idxmax().strftime('%Y-%m-%d')})")
95 print(f"    Min DCC corr       : {rho_series.min():.4f}
96       ({rho_series.idxmin().strftime('%Y-%m-%d')})")
97 print(f"    % time corr > 0.1 : {(rho_series >
98       0.1).mean()*100:.1f}%")

```

② Plain English

DCC captures that the BTC-Gold correlation is not static — it shifts over time with market regime. During crisis periods (COVID March 2020, FTX collapse November 2022), correlations spike as panic selling hits all assets simultaneously (“everything sells off”). During bull markets, correlations may turn negative as BTC rises on crypto-specific factors while Gold stagnates. Static correlation (one number) hides all this rich dynamics.

③ Common Mistake

Using rolling window correlation (e.g. `returns.rolling(30).corr()`) as a substitute for DCC. Rolling correlation treats all 30 observations equally, while DCC/EWMA exponentially downweights older observations. DCC is more responsive to recent regime changes and theoretically grounded in the GARCH framework.

④ Real World Link

Portfolio managers holding both BTC and Gold use DCC to dynamically adjust hedge ratios. When DCC correlation spikes positive (both assets volatile together), holding Gold as a “BTC hedge” provides no diversification benefit. BlackRock’s risk models use DCC-type models for exactly this cross-asset correlation management.

M4.027 *Predict the Output* **HARD****Topic:** ARCH LM test — before and after GARCH fitting

Without running this code, predict the qualitative outcome of each test and explain why they should differ.

```

1 from arch import arch_model, arch_lm_test
2 import numpy as np
3
4 np.random.seed(0)
5 omega, alpha, beta = 0.1, 0.1, 0.85
6 n = 1000
7 sigma2 = np.zeros(n); eps = np.zeros(n)
8 sigma2[0] = omega / (1 - alpha - beta)
9 for t in range(1, n):
10     sigma2[t] = omega + alpha*eps[t-1]**2 +
11         beta*sigma2[t-1]
12     eps[t] = np.sqrt(sigma2[t]) * np.random.randn()
13
14 returns = eps * 2
15
16 # ARCH LM test on RAW returns
17 lm_raw = arch_lm_test(returns, nlags=10)
18 print(f"LM test on raw returns: stat={lm_raw.stat:.4f},
19       p={lm_raw.pvalue:.6f}")
20
21 # Fit GARCH(1,1)
22 fit = arch_model(returns, mean="Zero", vol="GARCH", p=1,
23                 q=1).fit(dis="off")
24
25 # ARCH LM test on STANDARDISED residuals
26 std_resid = fit.std_resid.dropna()
27 lm_resid = arch_lm_test(std_resid, nlags=10)
28 print(f"LM test on std resid : stat={lm_resid.stat:.4f},
29       p={lm_resid.pvalue:.6f}")
30 print(f"GARCH removed ARCH effects? {lm_resid.pvalue >
31       0.05}")

```

Predict: (i) raw returns p-value direction, (ii) standardised residuals p-value direction, (iii) final conclusion.

Solution**① Predicted Output**

```

LM test on raw returns:  stat=[large], p=[very small, approx
0.000000]
LM test on std resid :  stat=[small], p=[large, > 0.05, perhaps
0.2--0.8]
GARCH removed ARCH effects?  True

```

```

1 # DATA GENERATION:
2 # Returns generated by GARCH(1,1) with alpha=0.1, beta=0.85
3 # Strong volatility clustering by construction
4 # alpha + beta = 0.95 -> high persistence -> slow decay
5
6 # LM TEST ON RAW RETURNS:
7 # Squared returns  $r^2_t$  are autocorrelated (defining ARCH
  property)
8 # With  $n=1000$  and genuine GARCH structure, LM test has
  enormous power
9 # -> Large test statistic >> critical value
10 # -> p-value  $\approx 0$  -> REJECT  $H_0$  -> ARCH detected
11
12 # GARCH FITTING:
13 # Fitted GARCH estimates  $\sigma^2_t \approx \text{true } \sigma^2_t$  for
  large  $n$ 
14 # Standardised residuals  $z_t = \varepsilon_t / \sigma_t \approx \text{iid}$ 
  standard normal
15
16 # LM TEST ON STANDARDISED RESIDUALS:
17 #  $z_t$  should be approximately iid (no autocorrelation in
   $z^2_t$ )
18 # -> Small test statistic (inside critical region)
19 # -> Large p-value -> FAIL to reject -> no ARCH in
  residuals
20 # -> GARCH successfully captured all variance dynamics

```

② Plain English

This two-step test is the GARCH equivalent of the Ljung-Box test before/after ARIMA fitting. Before fitting: LM test detects the ARCH effects we know are in the data ($p \approx 0$). After fitting: standardised residuals $z_t = \varepsilon_t / \sigma_t$ should be close to iid — GARCH “explains” the variance clustering, leaving behind clean residuals (large p-value).

③ Common Mistake

Testing ARCH LM on raw residuals (ε_t) rather than standardised residuals ($z_t = \varepsilon_t / \sigma_t$) after GARCH fitting. Raw residuals still have ARCH effects (the whole point of GARCH was to model their variance). Only standardised residuals — which divide out the estimated conditional volatility — should be white noise if the model is correct.

④ Real World Link

This is the standard model validation procedure for GARCH in professional practice. A risk model submission must show: (1) ARCH LM on raw returns → significant (justifying GARCH); (2) ARCH LM on standardised residuals → not significant

(confirming GARCH adequacy). Missing step 2 is a common failure in model validation reviews.

M4.028 *Fill in the Blank* **HARD**

Topic: GARCH with Student-t — comparing tail distributions

Fill in the blanks to compare BTC tail risk under Normal vs Student-t GARCH, and compute the difference in 99.9% VaR.

```

1 from arch import arch_model
2 import scipy.stats as stats
3
4 btc      = yf.download("BTC-USD", start="2019-01-01",
5                       end="2024-12-31")
6 returns  = np.log(btc["Close"]).diff().dropna() * 100
7
8 fit_n    = arch_model(returns, mean="Constant",
9                       vol="GARCH", p=1, q=1,
10                      dist="_____").fit(dis="off")
11
12 fit_t    = arch_model(returns, mean="Constant",
13                       vol="GARCH", p=1, q=1,
14                      dist="_____").fit(dis="off")
15
16 nu       = fit_t.params["_____"]
17 sigma_t  = fit_t.conditional_volatility.iloc[-1]
18 mu_t     = fit_t.params["mu"]
19
20 # 99.9% VaR (1 in 1000 day event)
21 q_normal = stats.norm.ppf(_____) # 0.1th percentile
22         of normal
23 q_student = stats.t.ppf(_____, df=nu) # 0.1th percentile
24         of t(nu)
25
26 var_normal = mu_t + q_normal * sigma_t
27 var_student = mu_t + q_student * sigma_t
28
29 print(f"Normal quantile (0.1%) : {q_normal:.4f}")
30 print(f"Student-t quantile(0.1%): {q_student:.4f}
31       (nu={nu:.2f})")
32
33 print(f"\nNormal    99.9% VaR : {var_normal:.4f}%")
34 print(f"Student-t  99.9% VaR: {var_student:.4f}%")
35 print(f"Fat-tail premium : {_____.4f}pp")
36 print(f"t-VaR is {_____.1f}x worse than Normal VaR")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, scipy.stats as stats
3 import warnings; warnings.filterwarnings('ignore')
4 from arch import arch_model
5
6 btc      = yf.download("BTC-USD", start="2019-01-01",
7                       end="2024-12-31")
8 returns  = np.log(btc["Close"]).diff().dropna() * 100
9
10 fit_n    = arch_model(returns, mean="Constant",
11                      vol="GARCH", p=1, q=1,
12                      dist="normal").fit(dispatch="off")
13
14 fit_t    = arch_model(returns, mean="Constant",
15                      vol="GARCH", p=1, q=1,
16                      dist="t").fit(dispatch="off")
17
18 nu       = fit_t.params["nu"]           # correct parameter
19         name
20 sigma_t  = fit_t.conditional_volatility.iloc[-1]
21 mu_t     = fit_t.params["mu"]
22
23 # 99.9% VaR: 0.1th percentile
24 q_normal  = stats.norm.ppf(0.001)       # z =
25         -3.09
26 q_student = stats.t.ppf(0.001, df=nu)   # more
27         negative than normal
28
29 var_normal = mu_t + q_normal * sigma_t
30 var_student = mu_t + q_student * sigma_t
31
32 fat_tail_premium = var_student - var_normal #
33         negative = t-VaR is worse
34 ratio          = abs(var_student) / abs(var_normal)
35
36 print(f"Student-t nu: {nu:.4f}")
37 print(f"Latest conditional vol: {sigma_t:.4f}/day")
38 print(f"\nQuantiles at 0.1th percentile:")
39 print(f"  Normal quantile      : {q_normal:.4f}")
40 print(f"  Student-t quantile: {q_student:.4f} (more
41     extreme -> worse VaR)")
42
43 print(f"\n99.9% VaR (one-in-1000-day loss):")
44 print(f"  Normal GARCH      : {var_normal:.4f}%")
45 print(f"  t-GARCH            : {var_student:.4f}%")
46 print(f"  Fat-tail premium: {fat_tail_premium:.4f}pp")
47 print(f"  t-VaR is {ratio:.2f}x worse than Normal VaR")
48 print(f"\nConclusion: Normal distribution understates BTC
49     tail risk by {ratio:.2f}x")
50 print(f"AIC comparison: Normal={fit_n.aic:.2f},
51     t-dist={fit_t.aic:.2f}")

```



```
40 print(f"t-dist wins: {fit_t.aic < fit_n.aic}")
```

② Plain English

The t-distribution has heavier tails than the normal — at the 0.1th percentile, the t-quantile is more extreme than the normal quantile, especially for small ν . For BTC with $\nu \approx 5$, the t-VaR at 99.9% is typically 1.3–1.7 $\times$ worse than Normal VaR. This means if a bank uses Normal-GARCH for BTC risk, it underestimates the true worst-case loss by 30–70%.

③ Common Mistake

Using `stats.t.ppf(0.001, df=nu)` vs `stats.norm.ppf(0.001)` without noting that both return negative numbers (left tail percentiles). The formula $\text{VaR} = \mu + q \times \sigma$ already produces a negative number (loss = negative return).

④ Real World Link

The 2008 financial crisis was partly caused by banks using Normal-distribution risk models that dramatically underestimated tail losses. For BTC specifically, the FTX collapse (November 2022) produced a 2-day loss of $\sim 30\%$ — an event that Normal-GARCH would have assigned probability $< 0.0001\%$, while t-GARCH with $\nu \approx 5$ might assign $\sim 0.01\%$. A 100 $\times$ difference in assessed probability drives vastly different capital requirements.

M4.029 Short Answer **HARD**

Topic: GARCH persistence and half-life

For a GARCH(1,1) with $\alpha = 0.10$, $\beta = 0.88$, derive the approximate “half-life” of a volatility shock — the number of days it takes for the volatility to decay halfway back to its long-run level — and interpret what this means for BTC risk management.

Solution

① Answer and Derivation

```
1 import numpy as np
2
3 alpha = 0.10; beta = 0.88
4 persistence = alpha + beta    # = 0.98
5
6 # Volatility shock decays at rate (alpha+beta) per period
7 # After h periods: remaining fraction = (alpha+beta)^h
8 # Half-life: (alpha+beta)^h = 0.5 -> h = log(0.5) /
   log(alpha+beta)
9
10 half_life = np.log(0.5) / np.log(persistence)
11
12 print(f"Persistence (alpha+beta) = {persistence:.2f}")
13 print(f"Half-life           = {half_life:.1f} trading
   days")
```

```

14 print(f"                                = {half_life/5:.1f}
    trading weeks")
15 print(f"                                = {half_life/21:.1f}
    trading months")
16
17 # Verification
18 for h in [5, 10, 20, 35, 50, 100]:
19     remaining = persistence**h * 100
20     print(f"    After {h:3d} days: {remaining:.1f}% of shock
    remains")

```

With $\alpha + \beta = 0.98$: $h = \log(0.5)/\log(0.98) \approx \mathbf{34.3 \text{ trading days}}$ (≈ 7 weeks).

Interpretation: A volatility shock to BTC (say the FTX collapse drove vol from 2% to 8% daily) would take approximately 34 trading days (7 weeks) to decay halfway back to the long-run level. After 100 days (~ 5 months), about 13% of the original shock would still remain. For BTC risk management: (i) increase VaR thresholds immediately after a major shock; (ii) don't reduce positions until at least 6–8 weeks have passed; (iii) long-dated BTC options should remain expensive for months after a crisis.

② Common Mistake

Computing half-life as $1/\log(\alpha + \beta)$ instead of $\log(0.5)/\log(\alpha + \beta)$. The correct formula comes from solving $(\alpha + \beta)^h = 0.5$, which gives $h = \log(0.5)/\log(\alpha + \beta)$. Both numerator and denominator are negative for $0 < \alpha + \beta < 1$, making h positive — always check the sign makes sense.

③ Real World Link

The 34-day half-life for BTC volatility explains why implied volatility on BTC options remains elevated for 6–8 weeks after major crashes. Options traders exploit this by selling volatility 2–3 weeks after a crash when implied vol is still elevated but the worst of the realized vol has passed. The half-life calculation tells them the timing precisely.

M4.030 Multiple Choice **HARD**

Topic: Johansen cointegration test

You run a Johansen cointegration test on the log price levels of BTC, Gold, and MRF (3 variables). The test returns: trace statistic = 42.3 for rank 0 (critical value: 29.8) and trace statistic = 11.2 for rank 1 (critical value: 15.5). What do you conclude?

- (A) No cointegration — all three assets are completely independent in the long run
- (B) One cointegrating relationship exists among the three price series
- (C) Two cointegrating relationships exist — two pairs of assets are cointegrated
- (D) Full cointegration — all three assets share the same long-run trend

Solution**① Correct Answer: (B)****② Plain English**

The Johansen trace test proceeds sequentially: first test H_0 : rank=0 (no cointegration). Trace statistic $42.3 > \text{critical value } 29.8 \rightarrow \text{reject } H_0 \rightarrow \text{at least 1 cointegrating vector exists}$. Then test H_0 : rank ≤ 1 . Trace statistic $11.2 < \text{critical value } 15.5 \rightarrow \text{fail to reject} \rightarrow \text{stop here}$. Conclusion: exactly 1 cointegrating relationship among BTC, Gold, and MRF prices.

③ Common Mistake

Stopping at the first test and concluding “cointegration exists” without specifying how many cointegrating vectors. The Johansen test is sequential — you must test rank=0, then rank=1, then rank=2 in order, stopping when you first fail to reject. Getting the rank wrong changes the VECM specification entirely.

④ Real World Link

If BTC, Gold, and MRF share one cointegrating vector, it means there’s a long-run price equilibrium they collectively mean-revert to — perhaps driven by a common macro factor like global dollar liquidity. A Vector Error Correction Model (VECM) would capture this: short-run VAR dynamics plus an “error correction” term that pulls prices back toward their long-run relationship whenever they deviate.

M4.031 Write from Scratch HARD

Topic: Comprehensive volatility regime analysis

Write a reusable function `volatility_regime_analysis(ticker, name)` that: (1) downloads data, computes returns, (2) fits GARCH(1,1) and t-GARCH(1,1), selects better by AIC, (3) classifies each day into volatility regime: Low (<25th pctile), Medium, High (>75th pctile), (4) computes regime statistics: mean return, std, % positive days per regime, (5) plots conditional volatility with regimes colour-coded, (6) returns regime classification Series.

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from arch import arch_model
6
7 def volatility_regime_analysis(ticker, name="Asset"):
8     """GARCH-based volatility regime analysis. Returns
9         (regime_series, best_fit)."""
10    data = yf.download(ticker, start="2019-01-01",
11                      end="2024-12-31", progress=False)

```

```

10     returns = np.log(data["Close"]).diff().dropna() * 100
11
12     # Fit Normal and t-GARCH, select by AIC
13     fit_n = arch_model(returns, mean="Constant",
14                        vol="GARCH", p=1, q=1,
15                        dist="normal").fit(dispatch="off")
16     fit_t = arch_model(returns, mean="Constant",
17                        vol="GARCH", p=1, q=1,
18                        dist="t").fit(dispatch="off")
19
20     if fit_t.aic < fit_n.aic:
21         best_fit = fit_t; dist_name = "t"; nu =
22             fit_t.params["nu"]
23     else:
24         best_fit = fit_n; dist_name = "Normal"; nu =
25             float("nan")
26
27     alpha = best_fit.params["alpha[1]"]; beta =
28         best_fit.params["beta[1]"]
29     omega = best_fit.params["omega"]; perst = alpha +
30         beta
31     hl = np.log(0.5) / np.log(perst)
32     lrvol = np.sqrt(omega / (1 - perst))
33
34     print(f"{'='*55}")
35     print(f" {name} ({ticker}) -- GARCH Regime Analysis")
36     print(f" Selected dist : {dist_name}
37         (AIC={best_fit.aic:.2f})")
38     print(f" alpha+beta : {perst:.4f} | Half-life:
39         {hl:.1f}d")
40     print(f" Long-run vol : {lrvol:.4f}%/day =
41         {lrvol*np.sqrt(252):.2f}%/yr")
42     if not pd.isna(nu): print(f" nu (t df) :
43         {nu:.2f}")
44
45     # Classify regimes
46     cond_vol = best_fit.conditional_volatility
47     q25 = cond_vol.quantile(0.25); q75 =
48         cond_vol.quantile(0.75)
49     regime = pd.Series("Medium", index=cond_vol.index,
50                        name="Regime")
51     regime[cond_vol <= q25] = "Low"
52     regime[cond_vol >= q75] = "High"
53
54     # Regime statistics
55     ret_aligned = returns.reindex(regime.index).dropna()
56     reg_aligned = regime.reindex(ret_aligned.index)
57     print(f"\n Regime Statistics:")

```

```

46     print(f"    {'Regime':8} {'Days':>6} {'Avg Ret':>10}
         {'Std Dev':>10} {'%Pos':>8}")
47     print(f"    {'-'*46}")
48     for reg in ["Low", "Medium", "High"]:
49         mask = reg_aligned == reg
50         r_reg = ret_aligned[mask]
51         print(f"    {reg:8} {len(r_reg):>6} "
52               f"{r_reg.mean():>+10.4f}% "
53               f"{r_reg.std():>10.4f}% "
54               f"{(r_reg>0).mean()*100:>7.1f}%")
55
56     # Plot
57     colors_map =
58         {"Low":"green","Medium":"orange","High":"red"}
59     fig, axes = plt.subplots(3, 1, figsize=(16, 12))
60     fig.suptitle(f"{name} -- GARCH Volatility Regime
61                 Analysis", fontsize=13)
62
63     # Returns coloured by regime
64     ax = axes[0]
65     for reg, col in colors_map.items():
66         mask = reg_aligned == reg
67         ax.scatter(ret_aligned[mask].index,
68                   ret_aligned[mask],
69                   c=col, s=4, alpha=0.6, label=reg)
70     ax.axhline(0, color="black", linewidth=0.8)
71     ax.set_title("Daily Returns by Regime");
72     ax.legend(fontsize=9, markerscale=3)
73
74     # Conditional volatility with thresholds
75     ax = axes[1]
76     ax.plot(cond_vol, color="navy", linewidth=0.8)
77     ax.axhline(q25, color="green", linewidth=1.2,
78               linestyle="--",
79               label=f"Low threshold: {q25:.2f}%")
80     ax.axhline(q75, color="red", linewidth=1.2,
81               linestyle="--",
82               label=f"High threshold: {q75:.2f}%")
83     ax.fill_between(cond_vol.index, cond_vol, q75,
84                     where=(cond_vol>=q75), color="red",
85                     alpha=0.3)
86     ax.fill_between(cond_vol.index, 0, cond_vol,
87                     where=(cond_vol<=q25), color="green",
88                     alpha=0.3)
89     ax.set_title(f"Conditional Volatility -- GARCH(1,1)
90                 {dist_name}")
91     ax.set_ylabel("Daily Vol (%)"); ax.legend(fontsize=9)
92
93     # Rolling regime mix

```

```

84     ax = axes[2]
85     high_ind =
        (regime=="High").astype(float).rolling(90).mean() *
        100
86     low_ind =
        (regime=="Low").astype(float).rolling(90).mean() *
        100
87     ax.fill_between(high_ind.index, high_ind, 0,
88                     color="red", alpha=0.5, label="%
        High-vol days (rolling 90d)")
89     ax.fill_between(low_ind.index, low_ind, 0,
90                     color="green", alpha=0.5, label="%
        Low-vol days (rolling 90d)")
91     ax.set_title("Rolling 90-Day Regime Mix");
        ax.set_ylim(0,100); ax.legend(fontsize=9)
92
93     plt.tight_layout(); plt.show()
94     print(f"{'='*55}\n")
95     return regime, best_fit
96
97 # Apply to all three course assets
98 btc_r, btc_fit = volatility_regime_analysis("BTC-USD",
        "Bitcoin")
99 mrf_r, mrf_fit = volatility_regime_analysis("MRF.NS",
        "MRF Tyres")
100 gold_r, gold_fit = volatility_regime_analysis("GC=F",
        "Gold Futures")

```

② Plain English

Volatility regime classification transforms the continuous GARCH conditional volatility into a discrete, interpretable signal. Low-regime days (calm markets) tend to show positive mean returns and high hit rates — consistent with the “volatility risk premium.” High-regime days show higher variance but ambiguous direction. This classification directly informs position sizing strategies.

③ Common Mistake

Using absolute volatility thresholds (e.g. “High = $\text{vol} > 5\%$ ”) instead of percentile-based thresholds. Absolute thresholds don’t adapt to different assets — a 5% daily vol is “High” for Gold but “Low” for BTC. Percentile-based thresholds ensure roughly 25%/50%/25% of days in each regime regardless of the asset’s absolute volatility level.

④ Real World Link

Volatility regime classification is the foundation of volatility-targeting strategies — systematic funds that scale position sizes inversely with volatility. When BTC enters a “High” regime, reduce BTC allocation by 50%; when in “Low” regime, increase to full allocation. This simple regime-following rule has been shown to improve risk-adjusted returns for BTC portfolios over 2019–2024.

M4.032 *Multiple Choice* **HARD****Topic:** BEKK GARCH — multivariate volatility

In a BEKK-GARCH model for the BTC-Gold system, the variance-covariance matrix evolves as: $H_t = C'C + A'\varepsilon_{t-1}\varepsilon'_{t-1}A + B'H_{t-1}B$. What is the key advantage of BEKK over modelling two separate GARCH(1,1) models?

- (A) BEKK always produces better AIC than two separate GARCHs
- (B) BEKK models the time-varying covariance between BTC and Gold, ensuring the covariance matrix H_t is positive definite at every time step
- (C) BEKK is computationally faster than two separate GARCHs
- (D) BEKK automatically selects the optimal lag orders p and q for each variable

Solution**① Correct Answer:** (B)**② Plain English**

Two separate GARCH(1,1) models capture each asset's own volatility dynamics but ignore how they co-move — the off-diagonal covariance $\sigma_{\text{BTC,Gold},t}$ is assumed constant. BEKK models the full 2×2 covariance matrix H_t jointly, ensuring: (1) variances are always positive; (2) the correlation between BTC and Gold is time-varying and can spike during crises; (3) the full matrix H_t is positive definite at every time step by construction. This matters for portfolio VaR, which requires a valid covariance matrix.

③ Common Mistake

Thinking DCC (Dynamic Conditional Correlation) and BEKK are equivalent. Both model time-varying covariances but differently. BEKK directly models the covariance matrix; DCC first standardises residuals then models the correlation matrix separately. BEKK is more general but has more parameters. DCC is more parsimonious and scales better to larger systems.

④ Real World Link

Portfolio risk managers need the full time-varying covariance matrix for portfolio VaR calculations. If BTC-Gold covariance spikes from near-zero to positive during a market crisis (as happened in March 2020), the portfolio VaR based on separate GARCH models would underestimate risk because it misses the correlated crash. BEKK or DCC correctly captures this “correlation breakdown in crises.”

M4.033 *Predict the Output* **HARD****Topic:** GARCH persistence — simulation behaviour

Without running this code, predict qualitative behaviour of both series and explain the key difference.

```

1 from arch import arch_model
2 import numpy as np
3
4 np.random.seed(10)
```



```

5 omega1, alpha1, beta1 = 1.0, 0.05, 0.50    # System 1: Low
   persistence
6 omega2, alpha2, beta2 = 0.1, 0.08, 0.91    # System 2: High
   persistence
7
8 def simulate_garch(omega, alpha, beta, n=500):
9     sigma2 = np.zeros(n); eps = np.zeros(n)
10    sigma2[0] = omega / (1 - alpha - beta)
11    for t in range(1, n):
12        sigma2[t] = omega + alpha*eps[t-1]**2 +
            beta*sigma2[t-1]
13        eps[t] = np.sqrt(sigma2[t]) * np.random.randn()
14    return eps, np.sqrt(sigma2)
15
16 r1, vol1 = simulate_garch(omega1, alpha1, beta1)
17 r2, vol2 = simulate_garch(omega2, alpha2, beta2)
18
19 print(f"System 1 (a+b={alpha1+beta1:.2f}): vol
   mean={vol1.mean():.4f},"
20       f" vol std={vol1.std():.4f}")
21 print(f"System 2 (a+b={alpha2+beta2:.2f}): vol
   mean={vol2.mean():.4f},"
22       f" vol std={vol2.std():.4f}")
23 print(f"System 1 vol max/mean:
   {vol1.max()/vol1.mean():.2f}")
24 print(f"System 2 vol max/mean:
   {vol2.max()/vol2.mean():.2f}")

```

Predict: (i) which system's volatility will be more variable? (ii) which will have a higher max/mean ratio? (iii) which better represents BTC?

Solution

① Predicted Output

System 1 ($\alpha+\beta=0.55$): vol mean $\approx[1.4\text{--}1.6]$, vol std $\approx[0.2\text{--}0.4]$
(stable)
System 2 ($\alpha+\beta=0.99$): vol mean $\approx[2.0\text{--}2.5]$, vol std $\approx[0.8\text{--}2.0]$
(highly variable)
System 1 vol max/mean: $\approx 1.5\text{--}2.5$ (modest spikes)
System 2 vol max/mean: $\approx 4.0\text{--}10.0$ (dramatic spikes)

```

1 # System 1: alpha+beta = 0.55 -> LOW persistence
2 # Shock half-life ~ = 1.2 days -- vol recovers almost
   immediately
3 # Vol time series is stable, small excursions around
   long-run mean
4 # Low max/mean ratio -- spikes are brief and modest

```



```

5 # Long-run var = 1.0/(1-0.55) ~= 2.22 -> daily vol ~=
   1.49%
6
7 # System 2: alpha+beta = 0.99 -> HIGH persistence
8 # Shock half-life ~= 69 days -- shocks accumulate, slow
   decay
9 # Long quiet periods followed by explosive bursts
10 # High vol std AND high max/mean ratio
11 # Long-run var = 0.1/(1-0.99) = 10 -> daily vol ~= 3.16%
12 # (Higher long-run vol despite SMALLER omega! Due to
   high persistence)
13
14 # BTC matches System 2: alpha+beta ~= 0.95-0.99
15 # Calm periods of 1-2% daily vol followed by crisis
   periods of 5-10%
16 # persisting for months -- exactly System 2's behaviour
17 import numpy as np
18 print(f"Long-run vol System 1: {(1.0/0.45)**0.5:.3f}%")
19 print(f"Long-run vol System 2: {(0.1/0.01)**0.5:.3f}%")

```

② Plain English

High persistence ($\alpha + \beta$ close to 1) means volatility shocks have long memory — today's spike raises tomorrow's variance, which raises the day after. System 2 generates “volatility clusters” that can last weeks. System 1 forgets shocks within days. BTC's GARCH parameters ($\alpha + \beta \approx 0.95\text{--}0.99$) match System 2 — explaining why BTC volatility regimes last weeks to months.

③ Common Mistake

Predicting System 1 has higher average volatility because $\omega = 1.0$ is larger than $\omega = 0.1$. The long-run variance is $\omega/(1 - \alpha - \beta)$, not just ω . System 2: $0.1/(1 - 0.99) = 10$, daily vol = 3.16%. System 1: $1.0/(1 - 0.55) = 2.22$, daily vol = 1.49%. Counterintuitively, smaller ω with higher persistence produces higher long-run volatility.

④ Real World Link

This simulation demonstrates why GARCH persistence ($\alpha + \beta$) matters more than the absolute parameter values for risk management. A BTC model with $\alpha + \beta = 0.98$ will produce very different VaR estimates than one with $\alpha + \beta = 0.90$ — the near-integrated model generates occasional extreme vol regimes that dramatically affect tail risk.

M4.034 Short Answer **HARD**

Topic: Cointegration vs correlation in multivariate time series

In 2 sentences, explain the difference between correlation and cointegration for price

level time series like BTC and Gold, and state why cointegration is more economically meaningful for long-run relationship analysis.

Solution

① Model Answer

Correlation measures the contemporaneous linear co-movement of two stationary series at a point in time — but BTC and Gold prices are non-stationary (integrated), and the correlation between two random walks can appear high simply due to shared trending behaviour, not any genuine economic link (this is “spurious correlation”). Cointegration tests whether a specific linear combination of the non-stationary price series is stationary (a genuine long-run equilibrium), meaning the assets share a common stochastic trend and will not drift arbitrarily far apart — which is economically meaningful because it implies a structural relationship that can be exploited for pairs trading, hedge ratio estimation, and risk management.

② Common Mistake

Saying “correlation = 0.7 means the assets are cointegrated.” Correlation and cointegration are completely different concepts. High correlation between two trending price series can be entirely spurious — it would vanish once you look at returns (first differences). Cointegration is a property of the levels series and tests for a specific long-run equilibrium, not just contemporaneous co-movement.

③ Real World Link

BTC and Ethereum, sharing the same blockchain ecosystem, investor base, and sentiment drivers, are often found to be cointegrated. A pairs trading strategy exploiting BTC-ETH cointegration would short the “expensive” one and buy the “cheap” one relative to their long-run equilibrium ratio — a strategy that generated consistent returns during 2019–2023.

Section D ★ Exam Style Problems — M4.035 to M4.040

M4.035 Exam Style — Complete GARCH Analysis: BTC-USD EXAM

Marks: 20 | Time: 35 minutes | Dataset: BTC-USD

Part A — ARCH Effect Detection

[5 marks]

Download BTC data (2019–2024). Compute log returns ($\times 100$). Plot squared returns and ACF of squared returns. Run Engle’s LM test (10 lags). State null hypothesis, test result, and whether GARCH is justified.

Part B — GARCH(1,1) Fitting

[5 marks]

Fit GARCH(1,1) with Normal and Student-t innovations. Print both summaries. Compare AIC. Report: $\omega, \alpha, \beta, \alpha + \beta, \nu$ (if t), half-life of shock, long-run volatility.

Part C — Conditional Volatility and VaR

[5 marks]

Plot conditional volatility from the better model. Compute daily 99% VaR time

series. Count and report VaR violations. Determine if the model is conservative or aggressive.

Part D — Residual Validation

[5 marks]

Compute standardised residuals. Run ARCH LM test on them. Plot ACF of squared standardised residuals. State whether GARCH successfully captured the variance dynamics.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as stats
4 import warnings; warnings.filterwarnings('ignore')
5 from arch import arch_model, arch_lm_test
6 from statsmodels.graphics.tsaplots import plot_acf
7
8 btc      = yf.download("BTC-USD", start="2019-01-01",
9                       end="2024-12-31")
10 returns = np.log(btc["Close"]).diff().dropna() * 100
11
12 #          PART A -- ARCH Effect Detection [5 marks]
13
14 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
15 axes[0].fill_between(returns.index, returns**2, 0,
16                     color="orange", alpha=0.7)
17 axes[0].set_title("PART A -- Squared BTC Returns (clusters
18                   = ARCH)")
19 axes[0].set_ylabel("r^2 (%^2)")
20 plot_acf(returns**2, lags=30, ax=axes[1], zero=False,
21         alpha=0.05, color="orange")
22 axes[1].set_title("ACF of Squared Returns (spikes = ARCH
23                   effects)")
24 plt.tight_layout(); plt.show()
25
26 lm = arch_lm_test(returns, nlags=10)
27 print("PART A -- Engle's ARCH LM Test")
28 print(f"  H0: No ARCH effects (constant conditional
29       variance)")
30 print(f"  H1: ARCH effects present (volatility
31       clustering)")
32 print(f"  LM Statistic : {lm.stat:.4f}")
33 print(f"  p-value       : {lm.pvalue:.8f}")
34 print(f"  Decision: {'REJECT H0 -- ARCH effects confirmed'
35       if lm.pvalue<0.05 else 'Fail to reject'}")
36
37 #          PART B -- GARCH Fitting [5 marks]

```

```

30 fit_n = arch_model(returns, mean="Constant",
31                   vol="GARCH", p=1, q=1,
32                   dist="normal").fit(dispatch="off")
33 fit_t = arch_model(returns, mean="Constant",
34                   vol="GARCH", p=1, q=1,
35                   dist="t").fit(dispatch="off")
36
37 print("\nPART B -- Normal GARCH"); print(fit_n.summary())
38 print("\nPART B -- t-GARCH");      print(fit_t.summary())
39
40 best_fit = fit_t if fit_t.aic < fit_n.aic else fit_n
41 best_dist = "t" if fit_t.aic < fit_n.aic else "Normal"
42 alpha_b = best_fit.params["alpha[1]"]; beta_b =
43           best_fit.params["beta[1]"]
44 omega_b = best_fit.params["omega"]; perst = alpha_b +
45           beta_b
46 hl = np.log(0.5) / np.log(perst); lrvol = np.sqrt(omega_b
47           / (1 - perst))
48
49 print(f"\nBest model: {best_dist} GARCH
50       (AIC={best_fit.aic:.2f})")
51 print(f"    omega={omega_b:.6f}, alpha={alpha_b:.6f},
52       beta={beta_b:.6f}")
53 print(f"    alpha+beta={perst:.6f} | Half-life={hl:.1f}d |
54       LR vol={lrvol:.4f}%/day")
55 if best_dist == "t":
56     print(f"    nu={best_fit.params['nu']:.4f}")
57
58 #          PART C -- Conditional Vol and VaR [5 marks]
59
60 cond_vol = best_fit.conditional_volatility
61 mu        = best_fit.params["mu"]
62 var_99     = mu + stats.norm.ppf(0.01) * cond_vol
63
64 fig, axes = plt.subplots(2, 1, figsize=(16, 10))
65 axes[0].plot(cond_vol, color="purple", linewidth=0.8,
66              label=f"Conditional Vol ({best_dist} GARCH)")
67 axes[0].axhline(lrvol, color="red", linewidth=1.5,
68                linestyle="--",
69                label=f"Long-run vol = {lrvol:.2f}%")
70 axes[0].set_title(f"PART C -- BTC Conditional Volatility
71                  ({best_dist} GARCH(1,1))")
72 axes[0].legend()
73
74 violations = returns < var_99
75 axes[1].plot(returns.index, returns, color="steelblue",
76              linewidth=0.5, alpha=0.7, label="BTC Return")
77 axes[1].plot(var_99.index, var_99, color="red",
78              linewidth=1.5, label="99% VaR")

```

```

68 axes[1].scatter(returns[violations].index,
69               returns[violations],
70               color="darkred", s=15, zorder=5,
71               label="Violation")
72 axes[1].set_title("PART C -- VaR Violations");
73 axes[1].legend(fontsize=8)
74 plt.tight_layout(); plt.show()
75
76 n_viol = violations.sum(); viol_rate = violations.mean()
77 print(f"\nPART C -- VaR Backtest")
78 print(f"    Expected at 99%: {len(returns)*0.01:.1f} (1%)")
79 print(f"    Actual breaks   : {n_viol} | Rate:
80       {viol_rate*100:.2f}%")
81 print(f"    Verdict: {'Conservative' if viol_rate<0.01 else
82       'Aggressive -- underestimates risk'}")
83
84 #          PART D -- Residual Validation [5 marks]
85
86 std_resid = best_fit.std_resid.dropna()
87 lm_resid  = arch_lm_test(std_resid, nlags=10)
88
89 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
90 plot_acf(std_resid**2, lags=30, ax=axes[0], zero=False,
91         alpha=0.05)
92 axes[0].set_title("PART D -- ACF of Squared Std Residuals
93         (want: all inside bands)")
94 axes[1].hist(std_resid, bins=80, density=True,
95         color="purple", alpha=0.6)
96 x = np.linspace(std_resid.min(), std_resid.max(), 300)
97 axes[1].plot(x, stats.norm.pdf(x, 0, 1), "r-", lw=2,
98         label="N(0,1)")
99 axes[1].set_title("Std Residuals vs N(0,1)");
100 axes[1].legend()
101 plt.tight_layout(); plt.show()
102
103 print(f"\nPART D -- Residual ARCH LM:
104       p={lm_resid.pvalue:.4f}")
105 print(f"    Verdict: {'ARCH effects removed -- GARCH
106       well-specified' if lm_resid.pvalue>0.05 else 'Residual
107       ARCH -- consider GJR-GARCH or higher order'}")
108
109 print(f"\nFinal Assessment:")
110 print(f"    1. ARCH LM raw: p~0 -> GARCH justified")
111 print(f"    2. {best_dist} GARCH selected by AIC")
112 print(f"    3. alpha+beta={perst:.4f} -> {hl:.0f}-day shock
113       half-life")
114 print(f"    4. Residual LM p={lm_resid.pvalue:.4f} ->
115       {'adequate' if lm_resid.pvalue>0.05 else 'inadequate'}")

```

M4.036 Exam Style — VAR System: BTC, MRF, Gold EXAM**Marks: 20 | Time: 35 minutes | Dataset: All Three****Part A — Data Preparation and Stationarity** [4 marks]

Download all three assets (2020–2024). Compute log returns. Run ADF test on each. Create a 3-row stationarity summary table. Confirm all series are stationary before VAR.

Part B — VAR Lag Selection and Fitting [5 marks]

Build the 3-variable VAR. Use `select_order(maxlags=10)` and print results. Fit VAR with AIC-optimal lags. Print summary. Report number of parameters.

Part C — Granger Causality Matrix [6 marks]

Run Granger causality tests for all 6 directed pairs. Print a formatted 3×3 causality matrix with p-values. Identify which assets “lead” the system.

Part D — Impulse Response Functions [5 marks]

Compute and plot orthogonalised IRFs for all 9 combinations over 10 periods. Identify: which cross-asset effects are significant, and how long they persist.

Solution**① Complete Solution**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.vector_ar.var_model import VAR
6 from statsmodels.tsa.stattools import adfuller
7
8 tickers = {"BTC": "BTC-USD", "MRF": "MRF.NS", "Gold": "GC=F"}
9 rets = {}
10 for name, ticker in tickers.items():
11     raw = yf.download(ticker, start="2020-01-01",
12                     end="2024-12-31", progress=False)
13     rets[name] = np.log(raw["Close"]).diff().dropna()
14
15 data = pd.DataFrame(rets).dropna()
16 print(f"Aligned: {data.shape[0]} obs, {data.shape[1]} variables")
17
18 # PART A -- Stationarity [4 marks]
19
20 print("\nPART A -- ADF Stationarity Tests")
21 print("="*55)
22 print(f"   {'Asset':6} {'ADF Stat':>10} {'p-value':>10} {'Stationary?':>12}")
23 print(f"   {'-'*50}")
24 all_stat = True
25 for col in data.columns:

```

```

24     adf = adfuller(data[col])
25     yn  = "YES" if adf[1] < 0.05 else "NO"
26     if adf[1] >= 0.05: all_stat = False
27     print(f"    {col:6} {adf[0]:>10.4f} {adf[1]:>10.6f}
        {yn:>12}")
28 print(f"    All stationary (VAR ready): {all_stat}")
29
30 #          PART B -- VAR Lag Selection [5 marks]
31
32 model    = VAR(data)
33 lag_sel  = model.select_order(maxlags=10)
34 print(f"\nPART B -- Lag Selection");
35         print(lag_sel.summary())
36
37 aic_lag  = lag_sel.selected_orders["aic"]
38 bic_lag  = lag_sel.selected_orders["bic"]
39 print(f"AIC selects: {aic_lag} lags | BIC selects:
        {bic_lag} lags")
40
41 fit      = model.fit(aic_lag)
42 k, p     = data.shape[1], fit.k_ar
43 n_params = k**2 * p + k
44 print(f"VAR({p}) | Variables: {k} | Total parameters:
        {n_params}")
45 print(fit.summary())
46
47 #          PART C -- Granger Causality Matrix [6 marks]
48
49 cols = list(data.columns)
50 print(f"\nPART C -- Granger Causality Matrix (p-values * =
        significant at 5%)")
51 print("Row = Caused, Column = Causing | ---=self")
52 print("="*45)
53 gc_mat = pd.DataFrame(float("nan"), index=cols,
        columns=cols)
54 for caused in cols:
55     for causing in cols:
56         if caused != causing:
57             gc = fit.test_causality(caused=caused,
58                                     causing=[causing])
59             gc_mat.loc[caused, causing] = round(gc.pvalue,
60                                                 4)
61
62 print(f"{' ':6}", end="")
63 for c in cols: print(f"{c:>10}", end="")
64 print()
65 for caused in cols:
66     print(f"{caused:6}", end="")
67     for causing in cols:

```

```

63     val = gc_mat.loc[caused, causing]
64     if pd.isna(val): print(f"{'---':>10}", end="")
65     else:
66         m = "*" if val < 0.05 else " "
67         print(f"{val:>9.4f}{m}", end="")
68     print()
69
70 print("\nSignificant links:")
71 for caused in cols:
72     for causing in cols:
73         if caused != causing and
74             gc_mat.loc[caused,causing] < 0.05:
75                 print(f"    {causing} -> {caused}:
76                     p={gc_mat.loc[caused,causing]:.4f}")
77
78 #          PART D -- Impulse Response Functions [5 marks]
79
80 irf = fit.irf( periods=10)
81 irf.plot(orth=True, signif=0.1)
82 plt.suptitle("PART D -- Orthogonalised IRFs: BTC-MRF-Gold
83             VAR System\n"
84             "Column=shock | Row=response", fontsize=10)
85 plt.tight_layout(); plt.show()
86
87 print("\nCross-Asset IRF Summary (max |IRF| in first 5
88     periods):")
89 irf_data = irf.orth_irfs
90 print(f"    {'Shock':6} -> {'Response':6}    {'Max |IRF|':>12}
91         {'Notable?':>10}")
92 print("    " + "-"*45)
93 for j, sv in enumerate(cols):
94     for i, rv in enumerate(cols):
95         if i == j: continue
96         max_irf = np.max(np.abs(irf_data[:5, i, j]))
97         notable = max_irf > data[rv].std() * 0.3
98         print(f"    {sv:6} -> {rv:6}    {max_irf:>12.6f}    "
99             f"{'YES' if notable else 'no':>10}")

```

M4.037 Exam Style — GJR-GARCH Asymmetry and Risk Forecasting EXAM
Marks: 15 | Time: 25 minutes | Dataset: MRF.NS and BTC-USD

Part A (5 marks): Fit GJR-GARCH(1,1) to MRF returns (Normal dist) and BTC returns (t-dist). Extract and interpret gamma (leverage coefficient) for both. State whether each shows classical leverage or inverse leverage.

Part B (5 marks): For both assets, compute: (a) volatility impact of a positive 3% return, (b) volatility impact of a negative 3% return. Show the asymmetry ratio: negative/positive impact. Comment on which asset has stronger asymmetry.

Part C (5 marks): Generate 22-day (1-month) volatility forecast from both GJR-GARCH models. Plot forecast paths. Annualise the day-22 forecast. Compute the BTC-to-MRF forecast volatility ratio and interpret.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, matplotlib.pyplot as
  plt
3 import warnings; warnings.filterwarnings('ignore')
4 from arch import arch_model
5
6 def fit_gjr(ticker, name, dist="normal"):
7     raw = yf.download(ticker, start="2019-01-01",
8         end="2024-12-31", progress=False)
9     returns = np.log(raw["Close"]).diff().dropna() * 100
10    model = arch_model(returns, mean="Constant",
11        vol="GARCH", p=1, o=1, q=1, dist=dist)
12    fit = model.fit(dispatch="off")
13    return fit, returns
14
15 fit_mrf, ret_mrf = fit_gjr("MRF.NS", "MRF",
16     dist="normal")
17 fit_btc, ret_btc = fit_gjr("BTC-USD", "BTC", dist="t")
18
19 # PART A -- GJR-GARCH Fitting [5 marks]
20
21 print("PART A -- GJR-GARCH Parameter Summary")
22 for fit, name, dist in
23     [(fit_mrf, "MRF", "Normal"), (fit_btc, "BTC", "t-dist")]:
24     omega = fit.params["omega"]; alpha =
25         fit.params["alpha[1]"]
26     gamma = fit.params["gamma[1]"]; beta =
27         fit.params["beta[1]"]
28     print(f"\n{name} ({dist}): omega={omega:.4f},
29         alpha={alpha:.4f},
30         f" gamma={gamma:.4f}, beta={beta:.4f}")
31     if gamma > 0:
32         print(" Classical leverage: NEGATIVE shocks ->
33             MORE volatility (equity-like)")
34     elif gamma < 0:
35         print(" Inverse leverage: POSITIVE shocks -> MORE
36             volatility (crypto-like)")
37     else:
38         print(" Symmetric: gamma ~ 0")

```

```

30 # PART B -- Volatility Impact Asymmetry [5 marks]

31 print("\nPART B -- Asymmetry Analysis (3% shock
    magnitude)")
32 print("="*55)
33 shock_sq = 3.0 ** 2 # squared shock = 9
34
35 for fit, name in [(fit_mrf, "MRF"), (fit_btc, "BTC")]:
36     alpha = fit.params["alpha[1]"]; gamma =
        fit.params["gamma[1]"]
37     impact_pos = alpha * shock_sq
38     impact_neg = (alpha+gamma) * shock_sq
39     asymmetry = impact_neg / impact_pos
40
41     print(f"\n{name}: alpha={alpha:.4f},
        gamma={gamma:.4f}")
42     print(f" +3% shock impact: {impact_pos:.4f}")
43     print(f" -3% shock impact: {impact_neg:.4f}")
44     print(f" Asymmetry ratio : {asymmetry:.4f}")
45     stronger = "negative" if asymmetry > 1 else "positive"
46     print(f" {stronger.title()} shocks create
        {abs(asymmetry-1)*100:.1f}% more vol impact")
47
48 # PART C -- 22-Day Volatility Forecast [5 marks]

49 fc_mrf = fit_mrf.forecast(horizon=22, reindex=False)
50 fc_btc = fit_btc.forecast(horizon=22, reindex=False)
51
52 vol_fc_mrf = np.sqrt(fc_mrf.variance.iloc[0].values)
53 vol_fc_btc = np.sqrt(fc_btc.variance.iloc[0].values)
54
55 fig, ax = plt.subplots(figsize=(14, 5))
56 ax.plot(vol_fc_mrf, "b-o", markersize=4, linewidth=1.5,
    label="MRF GJR-GARCH Forecast")
57 ax.plot(vol_fc_btc, "r-s", markersize=4, linewidth=1.5,
    label="BTC GJR-GARCH Forecast")
58 ax.set_title("PART C -- 22-Day Volatility Forecast")
59 ax.set_xlabel("Days Ahead"); ax.set_ylabel("Daily Vol (%)")
60 ax.legend(); plt.tight_layout(); plt.show()
61
62 vol_mrf_ann = vol_fc_mrf[-1] * np.sqrt(252)
63 vol_btc_ann = vol_fc_btc[-1] * np.sqrt(252)
64 vol_ratio = vol_fc_btc[-1] / vol_fc_mrf[-1]
65
66 print(f"\nPART C -- Annualised Day-22 Forecast")
67 print(f" MRF : {vol_fc_mrf[-1]:.4f}%/day ->
    {vol_mrf_ann:.2f}% annualised")
68 print(f" BTC : {vol_fc_btc[-1]:.4f}%/day ->
    {vol_btc_ann:.2f}% annualised")

```

```

69 print(f"    BTC/MRF ratio: {vol_ratio:.2f}x")
70 print(f"    Interpretation: BTC is ~{vol_ratio:.1f}x more
    volatile than MRF over the next month.")
71 print(f"    BTC options/risk premiums should be
    ~{vol_ratio:.1f}x higher for equivalent notional.")

```

M4.038 Exam Style — GARCH-Based Risk Management: VaR and ES **EXAM**

Marks: 20 | Time: 40 minutes | Dataset: BTC-USD

Part A (5 marks): Fit GARCH(1,1) with t-distribution. Compute both 95% and 99% VaR time series using t-distribution quantiles. Plot all three (returns, 95% VaR, 99% VaR).

Part B (5 marks): Implement walk-forward VaR backtesting (rolling 252-day training window, min 252 days). Count violations for both 95% and 99% VaR. Compute violation rates. Apply the Kupiec POF test formally.

Part C (5 marks): Compute Expected Shortfall (ES/CVaR) at 99% from the GARCH model — the expected loss given you're already in the VaR violation region. Compare ES to VaR.

Part D (5 marks): Write a structured Risk Report as a comment block covering: model specification, VaR calibration result, ES finding, and regulatory compliance assessment (Basel III requires violation rate $\leq 1.5\%$ for internal models).

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as stats
4 import warnings; warnings.filterwarnings('ignore')
5 from arch import arch_model
6
7 btc      = yf.download("BTC-USD", start="2019-01-01",
8   end="2024-12-31")
9 returns = np.log(btc["Close"]).diff().dropna() * 100
10
11 fit = arch_model(returns, mean="Constant",
12   vol="GARCH", p=1, q=1,
13   dist="t").fit(dispatch="off")
14
15 nu = fit.params["nu"]; mu = fit.params["mu"]; cond_vol =
16   fit.conditional_volatility
17
18 #          PART A -- VaR Time Series [5 marks]

```

```

16 q_95 = stats.t.ppf(0.05, df=nu); q_99 = stats.t.ppf(0.01,
    df=nu)
17 var_95 = mu + q_95 * cond_vol;    var_99 = mu + q_99 *
    cond_vol
18
19 fig, ax = plt.subplots(figsize=(16, 6))
20 ax.plot(returns.index, returns, color="steelblue",
    linewidth=0.5, alpha=0.7, label="BTC Return")
21 ax.plot(var_95.index, var_95, color="orange",
    linewidth=1.2, label="95% VaR (t-GARCH)")
22 ax.plot(var_99.index, var_99, color="red", linewidth=1.5,
    label="99% VaR (t-GARCH)")
23 ax.fill_between(returns.index, returns, var_99,
    where=(returns<var_99),
24                 color="red", alpha=0.4, label="99% VaR
    Violation")
25 ax.axhline(0, color="black", linewidth=0.8)
26 ax.set_title("PART A -- BTC t-GARCH(1,1): Returns vs VaR
    Thresholds")
27 ax.set_ylabel("Return / VaR (%)"); ax.legend(fontsize=8)
28 plt.tight_layout(); plt.show()
29
30 print(f"PART A | nu={nu:.4f} | q95={q_95:.4f} |
    q99={q_99:.4f}")
31
32 #          PART B -- Walk-Forward Backtest + Kupiec [5
    marks]
33 ret_arr = returns.values; n_total = len(ret_arr)
34 var95_wf=[]; var99_wf=[]; act_wf=[]
35
36 for t in range(252, n_total):
37     train = ret_arr[max(0, t-252):t]
38     try:
39         f_wf = arch_model(pd.Series(train),
            mean="Constant",
40                             vol="GARCH", p=1, q=1,
            dist="t").fit(dis="off")
41         nu_wf = f_wf.params["nu"]; mu_wf =
            f_wf.params["mu"]
42         vol_wf = f_wf.conditional_volatility.iloc[-1]
43         var95_wf.append(mu_wf + stats.t.ppf(0.05,
            df=nu_wf)*vol_wf)
44         var99_wf.append(mu_wf + stats.t.ppf(0.01,
            df=nu_wf)*vol_wf)
45         act_wf.append(ret_arr[t])
46     except Exception: continue
47
48 act=np.array(act_wf); v95=np.array(var95_wf);
    v99=np.array(var99_wf)

```

```

49 viol_95 = act<v95; viol_99 = act<v99
50 rate_95 = viol_95.mean(); rate_99 = viol_99.mean()
51
52 def kupiec_pof(viol, target, n):
53     x = viol.sum(); T = n; p = target; ph = x/T
54     if ph in (0, 1): return float("nan"), float("nan")
55     lr =
56         -2*(x*np.log(p)+(T-x)*np.log(1-p)-x*np.log(ph)-(T-x)*np.log(1-ph))
57     return lr, 1-stats.chi2.cdf(lr, df=1)
58
59 k95 = kupiec_pof(viol_95, 0.05, len(act)); k99 =
60     kupiec_pof(viol_99, 0.01, len(act))
61 print(f"\nPART B -- Walk-Forward Backtest ({len(act)}
62     steps)")
63 print(f" 95% VaR: {viol_95.sum()} violations |
64     rate={rate_95*100:.2f}% (target 5%)")
65 print(f"  Kupiec LR={k95[0]:.4f}, p={k95[1]:.4f} -> {'OK'
66     if k95[1]>0.05 else 'Reject'}")
67 print(f" 99% VaR: {viol_99.sum()} violations |
68     rate={rate_99*100:.2f}% (target 1%)")
69 print(f"  Kupiec LR={k99[0]:.4f}, p={k99[1]:.4f} -> {'OK'
70     if k99[1]>0.05 else 'Reject'}")
71
72 #          PART C -- Expected Shortfall [5 marks]
73
74 nu_est = fit.params["nu"]; sigma_avg = cond_vol.mean()
75 t_a = stats.t.ppf(0.01, df=nu_est); pdf_val =
76     stats.t.pdf(t_a, df=nu_est)
77 ez_cond = -(pdf_val/0.01) * (nu_est+t_a**2)/(nu_est-1)
78 es_daily = mu + ez_cond * sigma_avg
79 emp_es    = returns[returns < var_99].mean()
80 var_avg   = var_99.mean()
81
82 print(f"\nPART C -- Expected Shortfall (99%)")
83 print(f"  Parametric ES: {es_daily:.4f}%/day")
84 print(f"  Empirical ES : {emp_es:.4f}%/day")
85 print(f"  Avg 99% VaR : {var_avg:.4f}%/day")
86 print(f"  ES/VaR ratio : {es_daily/var_avg:.3f}x")
87 print(f"  Expected loss when VaR is breached is
88     {es_daily/var_avg:.2f}x the VaR threshold")
89
90 #          PART D -- Risk Report [5 marks]
91
92 print(f"""
93 # RISK REPORT -- BTC-USD GARCH VaR MODEL
94 # =====
95 # MODEL: GARCH(1,1), Student-t dist, nu={nu:.2f}
96 #   alpha={fit.params['alpha[1]']:.4f},
97 #   beta={fit.params['beta[1]']:.4f}

```

```

86 #   alpha+beta =
      {fit.params['alpha[1]']+fit.params['beta[1]']:.4f}
87 #
88 # VaR CALIBRATION
89 #   99% VaR walk-forward rate: {rate_99*100:.2f}% (target:
      1.00%)
90 #   Kupiec p-value (99%): {k99[1]:.4f} -> {'Adequate' if
      k99[1]>0.05 else 'Rejected'}
91 #   95% VaR walk-forward rate: {rate_95*100:.2f}% (target:
      5.00%)
92 #   Kupiec p-value (95%): {k95[1]:.4f} -> {'Adequate' if
      k95[1]>0.05 else 'Rejected'}
93 #
94 # EXPECTED SHORTFALL
95 #   99% ES: {es_daily:.4f}%/day | ES/VaR =
      {es_daily/var_avg:.2f}x
96 #   Loss when VaR is breached is {es_daily/var_avg:.2f}x
      the VaR threshold.
97 #
98 # REGULATORY COMPLIANCE (Basel III)
99 #   Traffic light -- Green: <=4 violations per 250 days
      (<=1.6%)
100 #   Our 99% rate: {rate_99*100:.2f}% -> {'GREEN zone' if
      rate_99*250<=4 else ('YELLOW zone' if rate_99*250<=9
      else 'RED zone')}}
101 #
102 # RECOMMENDATION
103 #   Consider: (1) GJR-GARCH for asymmetric vol;
104 #   (2) DCC-GARCH for portfolio VaR; (3) Stressed VaR for
      tails.
105 # =====
106 """

```

M4.039 Exam Style — Multivariate VAR + GARCH Integrated Analysis **EXAM**
Marks: 20 | Time: 40 minutes | Dataset: BTC and Gold

Part A (5 marks): Build VAR(2) for BTC-Gold returns. Verify stationarity of VAR system (check characteristic roots). Plot eigenvalues — they must all lie inside the unit circle for stability.

Part B (5 marks): Fit GARCH(1,1) to VAR residuals of each variable separately. Compute DCC-style correlation as rolling 60-day correlation of standardised residuals. Plot.

Part C (5 marks): Compute the minimum variance portfolio weight for BTC given time-varying GARCH variances and DCC correlation. Plot optimal BTC weight over time.

Part D (5 marks): Generate 10-day ahead VAR mean forecast and GARCH

variance forecast. Construct 95% joint confidence ellipse for day-5 forecast. Interpret the ellipse shape.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.vector_ar.var_model import VAR
6 from arch import arch_model
7
8 btc = yf.download("BTC-USD", start="2020-01-01",
9   end="2024-12-31", progress=False)
10 gold = yf.download("GC=F", start="2020-01-01",
11   end="2024-12-31", progress=False)
12
13 data = pd.DataFrame({
14     "BTC" : np.log(btc["Close"]).diff() * 100,
15     "Gold": np.log(gold["Close"]).diff() * 100
16 }).dropna().loc[lamba df: df.index.isin(
17     btc["Close"].dropna().index &
18     gold["Close"].dropna().index)]
19 data = data.dropna()
20
21 # PART A -- VAR Stability [5 marks]
22
23 fit_var = VAR(data).fit(2)
24 roots = fit_var.roots
25 print("PART A -- VAR(2) Stability Check")
26 for i, r in enumerate(roots):
27     print(f" Root {i+1}: |lambda|={abs(r):.4f} "
28           f"{'Inside unit circle' if abs(r)<1 else 'OUTSIDE -- unstable!'}")
29 print(f" VAR stable: {fit_var.is_stable()}")
30
31 fig, ax = plt.subplots(figsize=(7, 7))
32 theta = np.linspace(0, 2*np.pi, 100)
33 ax.plot(np.cos(theta), np.sin(theta), "k--", lw=1.5,
34   label="Unit Circle")
35 ax.scatter(roots.real, roots.imag, color="red", s=80,
36   zorder=5, label="VAR Roots")
37 ax.axhline(0, color="gray", lw=0.8); ax.axvline(0,
38   color="gray", lw=0.8)
39 ax.set_xlim(-1.5,1.5); ax.set_ylim(-1.5,1.5);
40 ax.set_aspect("equal")

```

```

33 ax.set_title("PART A -- VAR(2) Roots (all inside unit
    circle -> stable)")
34 ax.legend(); ax.grid(alpha=0.3); plt.tight_layout();
    plt.show()
35
36 #          PART B -- GARCH on VAR Residuals + DCC [5 marks]
37 r_btc  = pd.Series(fit_var.resid[:,0],
    index=data.index[fit_var.k_ar:])
38 r_gold = pd.Series(fit_var.resid[:,1],
    index=data.index[fit_var.k_ar:])
39
40 g_btc  = arch_model(r_btc, mean="Zero",
    vol="GARCH",p=1,q=1).fit(dis="off")
41 g_gold = arch_model(r_gold, mean="Zero",
    vol="GARCH",p=1,q=1).fit(dis="off")
42
43 z_btc  = g_btc.std_resid.dropna(); z_gold =
    g_gold.std_resid.dropna()
44 z_df   =
    z_btc.to_frame("z_BTC").join(z_gold.to_frame("z_Gold"),how="inner").d
45
46 roll_corr = z_df["z_BTC"].rolling(60).corr(z_df["z_Gold"])
47 static_c  = z_df.corr().loc["z_BTC","z_Gold"]
48
49 fig, axes = plt.subplots(2, 1, figsize=(14, 8))
50 axes[0].plot(g_btc.conditional_volatility, label="BTC
    GARCH Vol")
51 axes[0].plot(g_gold.conditional_volatility, label="Gold
    GARCH Vol")
52 axes[0].set_title("PART B -- GARCH Conditional Vol of VAR
    Residuals"); axes[0].legend()
53
54 axes[1].plot(roll_corr, color="purple", lw=1.2,
    label="60-Day Rolling DCC Corr")
55 axes[1].axhline(static_c, color="red", linestyle="--",
    label=f"Static rho={static_c:.4f}")
56 axes[1].axhline(0, color="black", lw=0.8)
57 axes[1].set_title("PART B -- Dynamic BTC-Gold
    Correlation"); axes[1].legend()
58 plt.tight_layout(); plt.show()
59
60
61 #          PART C -- Min-Var Portfolio Weight [5 marks]
62 all_aligned = pd.DataFrame({
63     "vol_btc" : g_btc.conditional_volatility,
64     "vol_gold": g_gold.conditional_volatility,
65     "rho"     : roll_corr
66 }).dropna()

```



```

67
68 sb = all_aligned["vol_btc"]; sg = all_aligned["vol_gold"];
   rho = all_aligned["rho"]
69 w_btc = (sg**2 - rho*sb*sg) / (sb**2 + sg**2 - 2*rho*sb*sg)
70 w_btc = w_btc.clip(0, 1)
71
72 plt.figure(figsize=(14, 5))
73 plt.plot(w_btc, color="orange", lw=1.2)
74 plt.axhline(w_btc.mean(), color="red", linestyle="--",
75             label=f"Avg BTC weight: {w_btc.mean():.2%}")
76 plt.axhline(0.5, color="gray", linestyle=":", label="50%
   weight")
77 plt.title("PART C -- Min-Var BTC Weight in BTC-Gold
   Portfolio (Time-Varying)")
78 plt.ylabel("Optimal BTC Weight"); plt.ylim(0,1);
   plt.legend()
79 plt.tight_layout(); plt.show()
80
81 print(f"PART C | Mean BTC weight: {w_btc.mean():.2%} | "
82       f"Range: [{w_btc.min():.2%}, {w_btc.max():.2%}]")
83
84 #           PART D -- Joint Confidence Ellipse [5 marks]
85
86 fc_var = fit_var.forecast(y=data.values[-fit_var.k_ar:],
   steps=10)
87
88 fc5_btc = fc_var[4, 0]; fc5_gold = fc_var[4, 1]
89
90 fc_gb = g_btc.forecast(horizon=5, reindex=False)
91 fc_gg = g_gold.forecast(horizon=5, reindex=False)
92 std5_btc = np.sqrt(fc_gb.variance.iloc[0, 4])
93 std5_gold = np.sqrt(fc_gg.variance.iloc[0, 4])
94 rho_avg = roll_corr.dropna().mean()
95
96 chi2_95 = 5.991 # 95th percentile of chi^2(2)
97 t_ell = np.linspace(0, 2*np.pi, 200)
98 ell_x = np.sqrt(chi2_95) * std5_btc * np.cos(t_ell)
99 ell_y = np.sqrt(chi2_95) * std5_gold * np.sin(t_ell)
100 ell_x_rot = ell_x + rho_avg * ell_y
101
102 fig, ax = plt.subplots(figsize=(8, 8))
103 ax.plot(fc5_btc+ell_x_rot, fc5_gold+ell_y_rot, "r-", lw=2,
104         label="95% Joint CI Ellipse")
105 ax.scatter([fc5_btc], [fc5_gold], color="red", s=100,
106           zorder=5, label="Day-5 Forecast")
107 ax.axvline(fc5_btc, color="gray", lw=0.8, linestyle=":")
108 ax.axhline(fc5_gold, color="gray", lw=0.8, linestyle=":")
109 ax.set_xlabel("BTC 5-Day Return Forecast (%)")
110 ax.set_ylabel("Gold 5-Day Return Forecast (%)")

```

```

107 ax.set_title("PART D -- 95% Joint Confidence Ellipse:
      Day-5\nBTC-Gold VAR+GARCH System")
108 ax.legend(); ax.grid(alpha=0.3); plt.tight_layout();
      plt.show()
109
110 print(f"\nPART D -- Ellipse Interpretation:")
111 print(f"  Day-5 mean: BTC={fc5_btc:.4f}%,
      Gold={fc5_gold:.4f}%")
112 print(f"  Forecast std: BTC={std5_btc:.4f}%,
      Gold={std5_gold:.4f}%")
113 print(f"  BTC/Gold vol ratio: {std5_btc/std5_gold:.2f}x")
114 print(f"  DCC correlation: {rho_avg:.4f}")
115 print(f"  Shape: {'WIDE in BTC direction -- BTC far more
      uncertain' if std5_btc>std5_gold else 'balanced'}")

```

M4.040 Exam Style — Module 4 Integrated Capstone **EXAM**

Marks: 25 | Time: 55 minutes | Dataset: BTC, MRF, Gold

Part A (5 marks): For all three assets, fit t-GARCH(1,1). Create a unified volatility dashboard: 3-row subplot showing conditional volatility. Annotate COVID crash (Mar 2020) and FTX collapse (Nov 2022) on all three panels.

Part B (5 marks): Compare GARCH parameters across assets in a table: $\omega, \alpha, \beta, \alpha + \beta, \nu$, half-life, long-run vol. Identify: (a) highest persistence, (b) heaviest tails, (c) lowest long-run volatility.

Part C (5 marks): Fit a 3-variable VAR on log returns. Run complete Granger causality matrix. Produce FEVD table at horizon 10. Identify the most “exogenous” variable.

Part D (5 marks): Build a GARCH-based vol-targeting portfolio: at each rebalancing date (monthly), size positions inversely to conditional volatility. Compare cumulative return vs equal-weight portfolio.

Part E (5 marks): Write a 15-line integrated research summary covering: (1) volatility characteristics, (2) cross-asset VAR dynamics, (3) portfolio implications, (4) key model limitations, (5) extensions needed for full risk management.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from arch import arch_model
6 from statsmodels.tsa.vector_ar.var_model import VAR
7
8 assets = {"BTC": "BTC-USD", "MRF": "MRF.NS", "Gold": "GC=F"}

```

```

9 ret_dict = {}
10 for name, ticker in assets.items():
11     raw = yf.download(ticker, start="2019-01-01",
12                       end="2024-12-31", progress=False)
13     ret_dict[name] = np.log(raw["Close"]).diff().dropna()
14     * 100
15
16 # PART A -- Unified Volatility Dashboard [5 marks]
17
18 garch_fits = {}
19 fig, axes = plt.subplots(3, 1, figsize=(18, 14))
20 colors_a = {"BTC": "orange", "MRF": "steelblue", "Gold": "gold"}
21 events = {"COVID (Mar '20)": "2020-03-16", "FTX
22           (Nov '22)": "2022-11-08"}
23
24 for ax, (name, returns) in zip(axes, ret_dict.items()):
25     fit = arch_model(returns, mean="Constant",
26                     vol="GARCH", p=1, q=1,
27                     dist="t").fit(dis="off")
28     garch_fits[name] = fit; cond_vol =
29         fit.conditional_volatility
30     ax.fill_between(cond_vol.index, cond_vol, 0,
31                    color=colors_a[name], alpha=0.5)
32     ax.plot(cond_vol, color=colors_a[name], lw=0.8)
33     ax.axhline(cond_vol.mean(), color="red", lw=1.5,
34               linestyle="--",
35               label=f"Avg: {cond_vol.mean():.2f}%")
36     for ev_name, ev_date in events.items():
37         ax.axvline(pd.Timestamp(ev_date), color="black",
38                   lw=1.5, linestyle=":")
39         ax.annotate(ev_name, xy=(pd.Timestamp(ev_date),
40                                cond_vol.max()*0.8),
41                    xytext=(5,0), textcoords="offset
42                            points", fontsize=8)
43     ax.set_title(f"{name} -- t-GARCH(1,1) Conditional
44                 Volatility (%/day)")
45     ax.set_ylabel("Daily Vol (%)"); ax.legend(fontsize=9)
46 fig.suptitle("PART A -- Volatility Dashboard: BTC, MRF,
47             Gold", fontsize=14)
48 plt.tight_layout(); plt.show()
49
50 # PART B -- Parameter Comparison [5 marks]
51
52 rows = []
53 for name, fit in garch_fits.items():
54     omega=fit.params["omega"]; alpha=fit.params["alpha[1]"]
55     beta=fit.params["beta[1]"]; nu=fit.params["nu"]
56     perst=alpha+beta; hl=np.log(0.5)/np.log(perst)
57     lrvol=np.sqrt(omega/(1-perst))

```

```

45     rows.append({"Asset":name, "omega":round(omega,6), "alpha":round(alpha,
46                  "beta":round(beta,4), "alpha+beta":round(perst,4),
47                  "nu":round(nu,2), "Half-life(d)":round(hl,1),
48                  "LR Vol(€/d)":round(lrvol,4)})
49
50 param_df = pd.DataFrame(rows).set_index("Asset")
51 print("PART B -- GARCH Parameter Comparison Table")
52 print("="*75); print(param_df.to_string()); print("="*75)
53 print(f"   Highest persistence :
54       {param_df['alpha+beta'].idxmax()} -> slowest vol decay")
55 print(f"   Heaviest tails      : {param_df['nu'].idxmin()}
56       -> most fat-tailed")
57 print(f"   Lowest LR vol       : {param_df['LR
58       Vol(€/d)'].idxmin()} -> calmest asset")
59
60 #           PART C -- VAR + Granger + FEVD [5 marks]
61
62 var_data = pd.DataFrame(ret_dict).dropna()
63 var_fit  = VAR(var_data).fit(maxlags=5, ic="aic")
64 cols     = list(var_data.columns)
65
66 # Granger matrix
67 gc_pvals = pd.DataFrame(float("nan"), index=cols,
68                          columns=cols)
69
70 for caused in cols:
71     for causing in cols:
72         if caused != causing:
73             gc = var_fit.test_causality(caused=caused,
74                                         causing=[causing])
75             gc_pvals.loc[caused,causing] =
76                 round(gc.pvalue, 4)
77
78 print("\nPART C -- Granger Causality Matrix (* =
79     significant at 5%)")
80
81 for i in cols:
82     for j in cols:
83         val = gc_pvals.loc[i,j]
84         if pd.isna(val): print(f"   --- ", end="")
85         else:
86             m = "*" if val < 0.05 else " "
87             print(f" {val:.3f}{m}", end="")
88     print(f" <- {i}")
89
90 # FEVD at horizon 10
91 fevd = var_fit.fevd(10)
92 fevd_10 = pd.DataFrame(fevd.decomp[9], index=cols,
93                        columns=cols)
94 print(f"\nFEVD at Horizon 10:")
95 print(fevd_10.round(4).to_string())

```

```

85 own_var = {c: fevd_10.loc[c,c] for c in cols}
86 most_exog = max(own_var, key=own_var.get)
87 print(f"\nMost exogenous: {most_exog}
      ({own_var[most_exog]*100:.1f}% own-shock explained)")
88
89 #          PART D -- Vol-Targeting Portfolio [5 marks]
90
91 returns_df = pd.DataFrame(ret_dict)
92 monthly_idx = returns_df.resample("ME").last().index
93 portfolio_vt=[]; portfolio_ew=[]
94
95 for i, date in enumerate(monthly_idx[1:], 1):
96     prev_date = monthly_idx[i-1]
97     month_ret =
98         returns_df.loc[(returns_df.index>prev_date)&(returns_df.index<=date)]
99     if len(month_ret) == 0: continue
100     vols = {}
101     for name, fit in garch_fits.items():
102         cv = fit.conditional_volatility
103         lv = cv[cv.index <= prev_date]
104         vols[name] = lv.iloc[-1] if len(lv) > 0 else 1.0
105     inv_vol = {k: 1/v for k, v in vols.items()}
106     total_iv = sum(inv_vol.values())
107     weights = {k: v/total_iv for k, v in inv_vol.items()}
108     month_means = month_ret.mean()
109     vt_ret = sum(weights[k]*month_means[k] for k in
110                 weights if k in month_means.index)
111     ew_ret = month_means.mean()
112     portfolio_vt.append(vt_ret);
113     portfolio_ew.append(ew_ret)
114
115 cum_vt = pd.Series(portfolio_vt).add(1).cumprod() - 1
116 cum_ew = pd.Series(portfolio_ew).add(1).cumprod() - 1
117 plt.figure(figsize=(14, 5))
118 plt.plot(cum_vt.values*100, label="GARCH Vol-Targeted",
119         color="purple", lw=2)
120 plt.plot(cum_ew.values*100, label="Equal-Weight",
121         color="steelblue", lw=2, linestyle="--")
122 plt.axhline(0, color="black", lw=0.8)
123 plt.title("PART D -- Cumulative Returns: Vol-Targeted vs
124         Equal-Weight Portfolio")
125 plt.ylabel("Cumulative Return (%)"); plt.legend();
126 plt.tight_layout(); plt.show()
127
128 sharpe_vt =
129     pd.Series(portfolio_vt).mean()/pd.Series(portfolio_vt).std()*np.sqrt(
130
131 sharpe_ew =
132     pd.Series(portfolio_ew).mean()/pd.Series(portfolio_ew).std()*np.sqrt(

```

```

122 print(f"\nPART D | Vol-Targeted:
      Return={cum_vt.iloc[-1]*100:.1f}%
      Sharpe={sharpe_vt:.2f}")
123 print(f"          | Equal-Weight:
      Return={cum_ew.iloc[-1]*100:.1f}%
      Sharpe={sharpe_ew:.2f}")
124
125 #          PART E -- Integrated Research Summary [5 marks]
126 btc_hl  = param_df.loc["BTC","Half-life(d)"]
127 mrf_hl  = param_df.loc["MRF","Half-life(d)"]
128 gold_hl = param_df.loc["Gold","Half-life(d)"]
129
130 print(f"""
131 # INTEGRATED RESEARCH SUMMARY -- T6725 Module 4 Capstone
132 # =====
133 # 1. VOLATILITY CHARACTERISTICS
134 #   BTC :
135 #       persistence={param_df.loc['BTC','alpha+beta']:.3f},
136 #       half-life={btc_hl:.0f}d,
137 #       nu={param_df.loc['BTC','nu']:.1f} ->
138 #       near-integrated vol, very heavy tails.
139 #   MRF :
140 #       persistence={param_df.loc['MRF','alpha+beta']:.3f},
141 #       half-life={mrf_hl:.0f}d.
142 #       Moderate persistence, equity-like dynamics.
143 #   Gold:
144 #       persistence={param_df.loc['Gold','alpha+beta']:.3f},
145 #       half-life={gold_hl:.0f}d.
146 #       Lowest long-run vol -- portfolio stabiliser.
147 #
148 # 2. CROSS-ASSET VAR DYNAMICS
149 #   VAR({var_fit.k_ar}) reveals limited Granger causality
150 #   at daily frequency.
151 #   Most exogenous: {most_exog}
152 #   ({own_var[most_exog]*100:.1f}% own-shock explained via
153 #   FEVD).
154 #   Assets largely driven by idiosyncratic shocks, not
155 #   cross-asset leads.
156 #
157 # 3. PORTFOLIO IMPLICATIONS OF GARCH VOL-TARGETING
158 #   Inverse-vol weighting auto-reduces BTC during crisis
159 #   periods.
160 #   Provides drawdown protection without market-timing
161 #   judgment.
162 #   Typically improves Sharpe ratio by reducing high-vol
163 #   exposure.
164 #
165 # 4. KEY MODEL LIMITATIONS

```

```

152 #     a) Univariate GARCH ignores cross-asset volatility
      spillovers.
153 #     b) VAR assumes linear, constant-parameter
      relationships.
154 #     c) GARCH stationarity may fail during structural
      breaks.
155 #     d) t-distribution may still underestimate BTC's
      extreme tails.
156 #
157 # 5. EXTENSIONS FOR FULL RISK MANAGEMENT
158 #     a) DCC-GARCH or BEKK for time-varying covariance
      matrix.
159 #     b) EGARCH or GJR-GARCH for asymmetric volatility
      responses.
160 #     c) Regime-switching GARCH for bull/bear market
      separation.
161 #     d) Extreme Value Theory (EVT) for tail risk beyond
      GARCH.
162 #     e) Johansen cointegration + VECM for long-run level
      relationships.
163 # =====
164 # """)

```

④ Real World Link

This capstone mirrors the analytical workflow of a multi-asset quantitative research team producing a quarterly risk report. Part E's integrated summary is the deliverable that reaches portfolio managers and risk committees — it synthesises statistical analysis into actionable intelligence. The ability to connect GARCH parameter estimates to investment implications (half-life $\rightarrow$ position holding period; $\nu \rightarrow$ VaR model choice; DCC $\rightarrow$ hedge ratio dynamics) is the defining skill of a senior quant in financial risk management.

Module 4 Complete

40 Problems — 10 Easy — 12 Medium — 12 Hard — 6 Exam

Range	Difficulty	Types
M4.001–M4.010	EASY	FIB, FTE, MCQ, PTO, SA
M4.011–M4.022	MEDIUM	FIB, FTE, GOR, MTF, MCQ, WFS
M4.023–M4.034	HARD	FIB, FTE, MCQ, WFS, PTO, SA
M4.035–M4.040	EXAM	Multi-part
Total	40 Problems	All 8 types

Problems	Type
M4.001, M4.002, M4.008, M4.011, M4.017, M4.021, M4.023, M4.028	Fill in the Blank
M4.007, M4.012, M4.018, M4.022, M4.024	Find the Error
M4.013	Get the Order Right
M4.014	Match the Following
M4.003, M4.004, M4.009, M4.015, M4.019, M4.025, M4.030, M4.032	Multiple Choice
M4.005, M4.027, M4.033	Predict the Output
M4.016, M4.020, M4.026, M4.031	Write from Scratch
M4.006, M4.010, M4.029, M4.034	Short Answer
M4.035–M4.040	Exam Style

Ready for **Module 5** — Integrated Projects: End-to-End Analysis on BTC-USD, MRF.NS, and GC=F